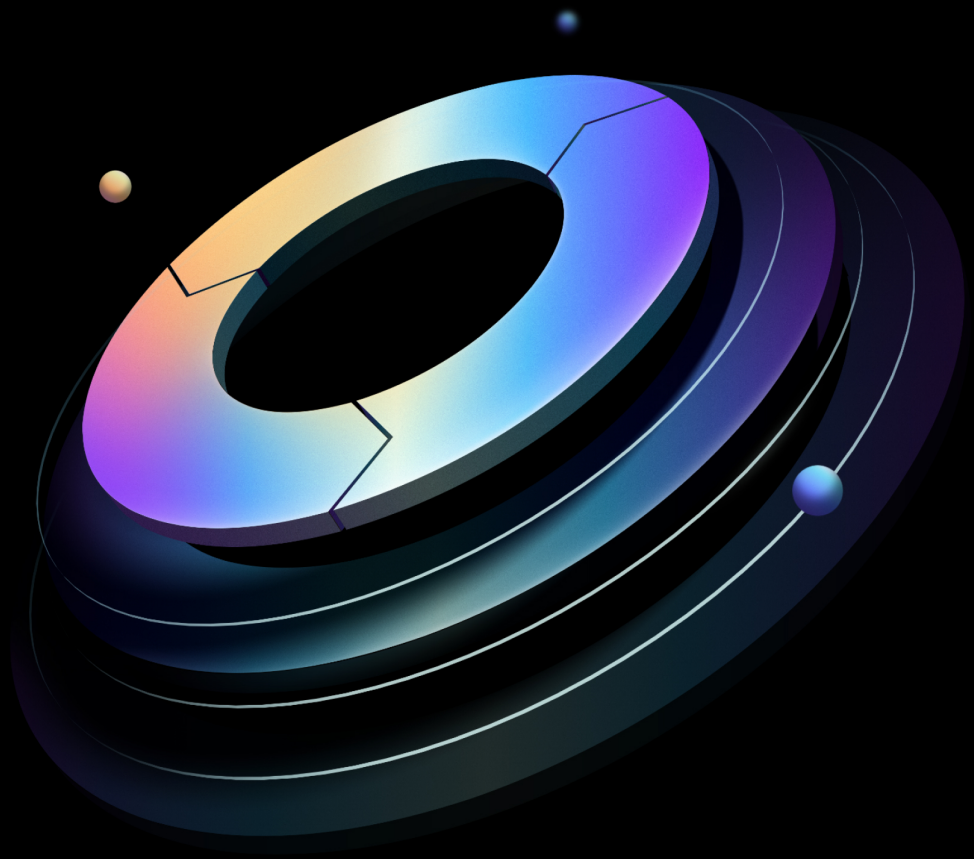
HashiCorp

Terraform User Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **4**
1.1 Why use HashiCorp Validated Designs? . . . 4
1.2 Audience . . . 4

2 IaC and cloud provisioning **5**
2.1 Landing zones . . . 7
2.1.1 Workload identity . . . 9

3 Terraform workflows **12**
3.1 Streamlining IaC . . . 12
3.1.1 Personas . . . 12
3.2 VCS-driven workflow . . . 13
3.2.1 Prerequisite(s) . . . 14
3.2.2 Workflow overview . . . 15
3.2.3 VCS integration . . . 16
3.2.4 VCS connection . . . 17
3.3 Auto apply . . . 18
3.4 Branching strategy . . . 19
3.4.1 Branching strategy for workspaces . . . 19
3.4.2 Branching strategy for Stacks . . . 28
3.5 Additional workflow options . . . 29
3.5.1 API-driven workflow . . . 30
3.5.2 Pipeline templates . . . 31
3.5.3 Command line-driven workflow . . . 33

4 Import existing resources **35**
4.1 Introduction . . . 35
4.2 People and process . . . 35
4.3 HashiCorp guidance . . . 36
4.3.1 What to import . . . 36
4.3.2 What not to import . . . 36
4.3.3 Following a phased approach . . . 37
4.3.4 A comparison of the two methods . . . 37

4.4 Using the Terraform `import` block . . . 38
4.4.1 Import block overview . . . 39
4.4.2 Building a configuration file . . . 39
4.4.3 Adding the `import` block . . . 40
4.4.4 Planning changes . . . 41
4.4.5 Applying changes . . . 42
4.4.6 Final steps . . . 44
4.4.7 Import block examples . . . 44
4.5 Using the Terraform import command . . . 55
4.5.1 Terraform import overview . . . 56
4.5.2 Import command examples . . . 56

5 Private Registry **57**
5.1 Discovering Terraform Modules and Providers . . . 58
5.2 Consuming Terraform Modules or Providers . . . 58

6 Policy as Code - Sentinel **59**
6.1 Explanation of the policy enforcement workflow . . . 59

7 Run Tasks **60**
7.1 Overall flow for Run Tasks within a run . . . 60
7.2 Available Run Tasks . . . 61

8 HCP Terraform Operator for Kubernetes **61**
8.0.1 HCP Terraform Operator Use Cases . . . 62
8.0.2 Auto-scaling agent pools for HCP Terraform . . . 62
8.0.3 Self-Service infrastructure via Kubernetes native consumption . . . 64
8.0.4 Installing the Operator . . . 66
8.0.5 Security considerations . . . 67
8.0.6 Observability . . . 70

9 Continuous validation **71**
9.1 Drift Detection . . . 71
9.1.1 Drift Resolution workflow . . . 73
9.2 **Continuous validation** . . . 75
9.3 Why Continuous Validation? . . . 75

9.4 Best Practice Recommendation . . . 75
9.5 Implementation guidance . . . 76
9.5.1 Requirements . . . 76
9.5.2 Permissions required . . . 76
9.5.3 Enabling health assessments . . . 77
9.5.4 Defining custom assertions and checks . . . 78
9.5.5 Authentication considerations . . . 78
9.5.6 Viewing health status and getting notified . . . 79

10 Terraform Workspace Notifications 79
10.1 Implementation guidance . . . 80
10.1.1 Permissions required . . . 81
10.1.2 Notification strategy . . . 81
10.1.3 Maintenance . . . 81

11 No-code provisioning 82

12 Ephemeral workspaces 82
12.1 Implementation guidance . . . 82
12.1.1 Roles and responsibilities . . . 82
12.1.2 Permissions requirements . . . 83
12.1.3 Configuring at scale . . . 83
12.1.4 Setting up notifications . . . 84

13 Changelog 84
13.1 2026-08 . . . 84
13.2 2026-01 . . . 84
13.3 2025-07 . . . 85
13.4 2025-06 . . . 85
13.5 2024-09 . . . 85
13.6 2024-04 . . . 85

14 Credits 85

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

The Terraform User Guide is for practitioners who consume HCP Terraform or Terraform Enterprise to provision and manage infrastructure. It assumes a Terraform service is available and that platform operators have configured the required organizations, projects, teams, workspaces, policy sets, registry content, integrations, and agent access.

Use this guide for consumption workflows:

- infrastructure as code and cloud provisioning
- Terraform workflows
- importing existing resources
- private-registry module and provider consumption

- policy-as-code enforcement during runs
- run-task results in Terraform runs
- HCP Terraform Operator for Kubernetes workflows
- continuous validation and health assessment
- workspace notifications
- no-code module consumption
- ephemeral workspaces

Note

Operator-owned setup and management for these capabilities belongs in the Terraform Administration Guide. Terraform Enterprise installation and deployment design belongs in the Terraform Installation Guide.

2 IaC and cloud provisioning

HCP Terraform provides you with a robust and efficient cloud provisioning workflow. The choice of workflow is important, and changing the workflow later requires engineering effort. The high level points required to implement a cloud provisioning workflow are as follows.

1. Plan your project, covering installation and configuration of HCP Terraform or Terraform Enterprise, and how you intend to provide self-service capability to your internal customers. The Terraform Administration and User Guides: Standardization HVD details this in more depth. Consideration now of how you onboard, and work with, early adopter teams, and later how to onboard gradually larger groups of teams until the platform is generally available saves a lot of time and effort.
2. Consider the size and bandwidth of your platform team. Consider what would happen to their workload if any of the onboarding experience of your internal customers was not fully automated.
3. Plan a landing zone provisioning workflow.
4. Ensure your platform team is fully trained on IaC with Terraform and the cloud provider(s) you are using. Expect all contributors to read, understand, and adhere to the HashiCorp Terraform language style guide. Use your documentation to guide all developers to write Terraform code that is compliant with this style guide.
5. Set up a Terraform Stack or workspace with cloud credentials.

6. Store your Terraform code in your strategic version control system.
7. Provision cloud resources from the IaC in the VCS using the Stack or workspace.
8. Consider the configuration required for HCP Terraform to enable the deployment end-to-end in the context of your organization's security and compliance requirements. Revisit each step in the workflow, and consider the requirements in terms of engineering content and effort, and focus on the manual steps and what you would need to do to automate each step. Also consider what would be necessary in a disaster recovery scenario.
9. If you intend for your deployment to scale to a large number of teams, ensure to read the respective HVDs in this series.

The landing zone provisioning workflow is a crucial process that you implement for your application or development teams to ensure infrastructure provisioning in the appropriate landing zones. Each cloud platform has its own recommendations for implementing these landing zones, and for Kubernetes-based teams, a Kubernetes namespace can also serve as a component of a landing zone.

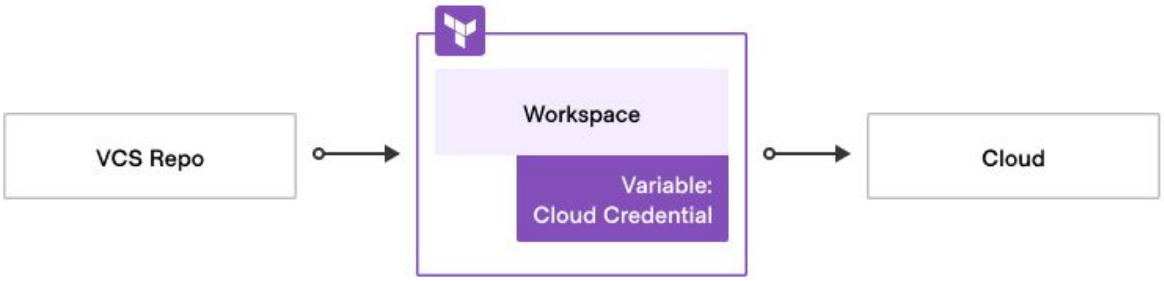


Figure 1: The cloud landing zone workspace

To achieve end-to-end cloud provisioning, developers commit Terraform code to the designated repository, triggering a plan in HCP Terraform which uses cloud credentials to complete the plan phase. Once approved, plan/apply the Stack or workspace(s), provisioning the desired resources in the cloud.

The platform team owns and implements a landing zone workflow, partnering the networking, security, and finance teams as necessary, adhering to the guidance provided by the cloud vendors' well-architected framework. Do this before any application teams can initiate infrastructure provisioning in the cloud.

2.1 Landing zones

A cloud landing zone is a foundational concept in cloud computing and a common deployment pattern for scaled customers. Defined as a standardized environment deployed in a cloud to facilitate secure, scalable, and efficient cloud operations, it forms a blueprint for setting up workloads in the cloud and serves as a launching pad for broader cloud adoption.

Cloud landing zones address several important areas:

- **Networking**: Configured with the network resources, such as virtual private clouds, subnets, and connectivity settings needed for operating workloads in the cloud.
- **Identity and access management**: Helps to enforce IAM policies, RBAC, and other permissions settings.
- **Security and compliance**: Security configurations such as encryption settings, security group rules, and logging settings.
- **Operations**: This includes operational tools and services, such as monitoring, logging, and automation tools. These help to manage and optimize cloud resources effectively.
- **Cost management**: Enables cost monitoring and optimization by facilitating automation, tagging policies, budget alerts, and cost reporting tools.

Major cloud providers offer their own solutions for this approach:

- AWS Control Tower
- Azure Landing Zone Accelerator
- Cloud Foundation Toolkit for GCP.

Landing zones are commonly deployed *by* HCP Terraform, but the landing zone concept is also applicable *to* HCP Terraform. The platform team deploys a control workspace during the onboarding of an internal customer using automation and which is then used to deploy the operational environment needed by that customer to support their development remit. This is akin to the equivalent CSP account and configuration needed by the same team. The recommended way to do this is as follows.

- **Complete preparative reading**: Read about cloud landing zone patterns applicable to the cloud service providers with whom you partner.
- **Define the HCP Terraform landing zone**: There are a number of ways to apply the LZ concept, but core requirements include:

    - Use of the Terraform Enterprise provider so that there is state representation for the configuration - this applies to both HCP Terraform and Terraform Enterprise.
    - Definition of a VCS template which contains boilerplate Terraform code for the LZ and a directory structure designed and managed by the platform team.
    - Creation of a Terraform module for your private registry - The platform team calls this through its automated onboarding process when an application team wants to onboard. This module nominally creates the following:
        - A control workspace for the application team.
        - A VCS repository for the application team to store the code which runs in the control workspace.
        - Variables/sets as needed for efficient deployment.
        - Public and private cloud resources for the team to use as necessary.
    - Augment your onboarding pipeline to make use of the module. Hook the process into other organizational platforms for credential generation and observability and so on.
    - Dedicate a HCP Terraform project to house the landing zone control workspaces separately from others the platform team own. Use the Terraform Enterprise provider or HCP Terraform API to deploy this. The onboarding process directs creation of control workspaces into this project.

- **Test**: Create landing zones using the process.
- **Update the module and VCS template**: **Expect to do this a number of times to refine the process** as you scale. Do not underestimate this key area. Depending on the onboarding design, if you onboard ten internal customers using the process, then need to change the process, this applies from that point on for the next set of customers. Depending on the implementation, you may need retrospective updates to refine the pipeline. This is also why it is important to funnel sets of early adopters onto the platform as you develop the solution using stepwise refinement.
- **Ensure smooth onboarding**: Work to ensure the onboarding process is as smooth as possible for the application teams. If using ServiceNow as front door to the onboarding process, liaise with the ServiceNow team *as early as possible* as we find there is always a lead time for remediation.

Summary of landing zone workflow

- **Ticket raised**: Audit trail created, approval acquired as appropriate.

- **Landing zone child module call code added**: The top-level workspace and its VCS repository is where the platform team collect and manage onboarded teams.
    - We recommend automating the addition of this child module call to ensure the solution is scalable. Use your strategic orchestration technology to manage this automation.
    - If your business expects to onboard thousands of teams, then avoid performance issues with the workspace run as discussed at the end of the previous section by placing significant consideration on prior planning of the platform team landing zone deployment workspace(s).
- **Run the top-level workspace**: This creates the control workspace for the application team using the Terraform landing zone module.
- **Application team control workspace created**: The template VCS contains Terraform code to use child modules to create the application team's Stacks or workspaces. In a franchise model, the application team would have write access to this repository so that they can configure it to deploy all the required Stacks or workspaces to support their needs. In a service catalog model, run the control workspace to create the application team's Stacks or workspaces to hook into subsequent request processes for pre-canned infrastructure requests.

Note

While we have found many of our scaled customers have derived significant success scaling Terraform onboarding through the use of the landing zone concept, each customer implements them differently, and the preceding advice is a generalization of the process. We recommend that you engage with a HashiCorp Solution Architect to discuss your specific requirements.

2.1.1 Workload identity

With dynamic credentials, users are able to authenticate their Vault providers using OpenID Connect (OIDC) compliant identity tokens to provide just-in-time credentials and eliminate the need to store long lived secrets on HCP Terraform/Enterprise. Some of the benefits of workload identity include the following.

- No more cloud credentials: Removes the need to store long-lived cloud credentials in HCP Terraform. Temporary cloud credentials are instead retrieved either from the CSP or using Vault on the fly.

- No more secret rotation logic: Credentials issued by cloud platform are temporary, removing the need to rotate secrets.
- Granular permissions control: Allows for using authentication and authorization tools from the cloud service provider to scope permissions based on HCP Terraform metadata such as organization, project, Stack or workspace, and run phase.

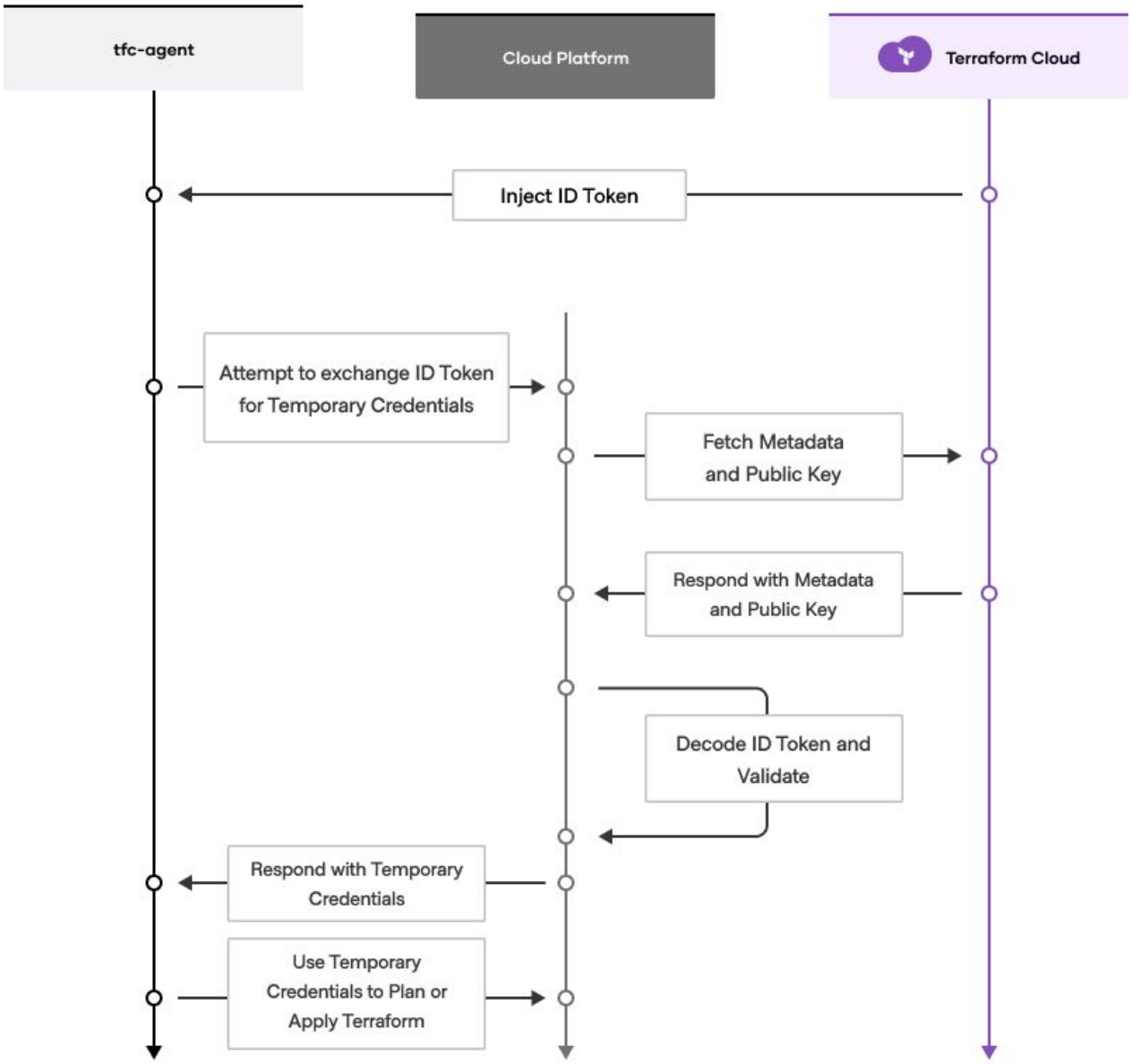


Figure 2: Workload identity with HCP Terraform

We recommend the implementation of workload identity to ensure that your cloud provisioning workflow adopts the most secure posture possible at all times. We expect you likely start by using fixed cloud credentials to authenticate your provider configuration. Include workflow identity in your

plans as the project proceeds and ensure the concept is well-understood and fully implemented prior to production deployment.

3 Terraform workflows

By adopting IaC practices, organizations have gained agility, efficiency, and reliability in deploying cloud-native resources. This section details integration of version control systems (VCS) and building efficient infrastructure deployment pipelines for Terraform.

3.1 Streamlining IaC

As you automate Infrastructure as Code (IaC) provisioning and management, you can streamline the process, similar to automating source code build and deployment with Continuous Integration/Continuous Deployment (CI/CD) pipelines. To simplify this automation and integrate with version control systems (VCS) repositories, HCP Terraform and Terraform Enterprise offer powerful features.

You can take advantage of the *VCS integration* feature in HCP Terraform and Terraform Enterprise, which uses webhooks to directly integrates with VCS repositories. This allows you to implement a GitOps workflow without the need for custom pipelines.

Most scaled customers have strategic CI/CD pipeline orchestrators. In situations where you prefer to utilize these, HCP Terraform and Terraform Enterprise form the infrastructure management component of such pipelines.

3.1.1 Personas

As with other recommendations in this document, we recommend the platform team designs and owns the infrastructure deployment pipeline design. Work with the following other personas to ensure efficiency of design end-to-end.

Persona	Role description	Role
Developer	Responsible for developing the infrastructure and application code.	Software engineer, application developer, consultant
Lead Developer	Responsible for helping the efforts of the product developers/teams.	Development lead, technical team Lead, head of development, technical manager
Release Engineer	Responsible for the coordination of a deployment to production (using automation)	Release engineer, release manager
Platform Engineer	Responsibilities include writing pipeline definitions and enabling developers to use the pipelines.	DevOps engineer, operations engineer, systems engineer, IT consultant
Infrastructure Operator	Responsibilities include maintenance, configuration and administration activities	Systems engineer, infrastructure engineer, site availability engineer, site reliability engineer, system administrator

3.2 VCS-driven workflow

When it comes to managing your infrastructure as code, Terraform offers a VCS-driven workflow that streamlines the process and enhances collaboration among your team members. This approach automatically triggers runs based on changes made in your VCS repositories, making it easier to keep track of modifications and ensuring that your infrastructure configuration stays up-to-date.

By establishing shared repositories as the definitive source of truth, your team can work concurrently on the same infrastructure code, promoting increased collaboration. You can also associate this

workflow with webhooks on your preferred VCS providers, streamlining the process of triggering runs and ensuring efficient synchronization between code changes and infrastructure updates.

3.2.1 Prerequisite(s)

Before proceeding with the VCS-driven workflow, ensure you complete the following pre-work for each category:

1. VCS repository:
    - Set up a Version Control System (VCS) repository that contains the source code for the deployment.
2. VCS authentication:
    - Establish VCS authentication enabling access to the repository securely for HCP Terraform.
3. VCS permissions:
    - Define the required VCS permissions for individuals, specifying appropriate access levels such as read-only, merge, and so on.
4. Deployment orchestration:
    - Choose whether to use workspaces or Stacks to orchestrate your Terraform deployments.

      If using workspaces:
        - Define workspace naming convention to maintain consistency.
        - Define workspace permissions, ensuring that the right users have the necessary access for collaboration.
        - Configure workspace settings, including enabling speculative plans, auto apply, and other relevant options.
        - Decide on a specific git tag format for auto apply to streamline the deployment process.

      If using Stacks:
        - Define Stack naming convention to maintain consistency.
        - Set permissions on the Stack at the project level

- Configure Stack settings, including enabling speculative plans, branch versus tag based VCS trigger, and other relevant options.

3.2.2 Workflow overview

These are the high level steps one would take to implement a successful VCS-driven workflow in Terraform.

1. Configure VCS integration: Establish a VCS integration for your organization to connect HCP Terraform and Terraform Enterprise with your VCS.
2. Connect workspace or Stack to repository: Link your workspace or Stack to the desired branch in your VCS repository, enabling automatic synchronization of code changes.
3. Adopt branching strategy: Adopt a suitable branching strategy, such as standard feature branching, to effectively manage different stages of development and deployment within your VCS repository.
4. Enable speculative plan runs: Automate speculative plan runs for each branch, allowing you to preview changes and validate configurations without impacting the live infrastructure.
5. Define PR process: Apply your organisation's standard PR process.
6. (optional) Configure automatic run triggers: Set up VCS run triggers for the plan phase based on git tag pushes, streamlining the deployment process and enhancing efficiency.

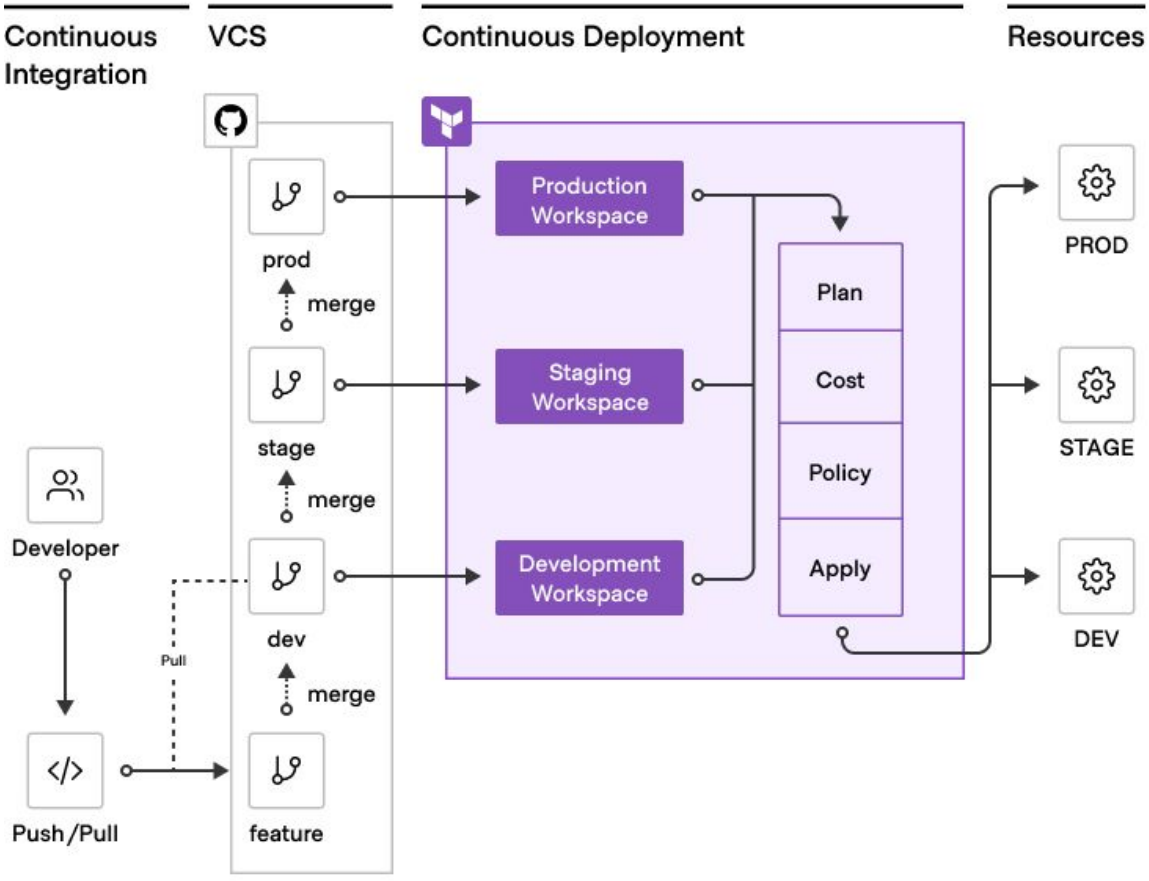


Figure 3: Workflow overview

3.2.3 VCS integration

Incorporating the VCS provider is crucial to establishing an automated workflow that predicts, plans, and initiates runs seamlessly. In the VCS-driven approach, a specific VCS repository backs each workspace or Stack. To access configurations from the VCS, Terraform requires permissions for the following tasks.

- Access a list of repositories: This enables you to search for repositories when creating new

workspaces or Stacks.
- Register webhooks with your VCS provider: Webhooks keep HCP Terraform informed about new commits to your chosen branch.
- Download repository contents at a specific commit: This action allows Terraform to execute with the specified code.

HCP Terraform uses webhooks to monitor new commits, pull requests and tags. An automatic run based on the configured repository branch triggers a workspace or Stacks run. Similarly, initiating a pull request or merge request activates speculative plans.

3.2.4 VCS connection

Integrating the VCS provider is essential for the workflow to work. HCP Terraform uses the OAuth protocol to authenticate with VCS providers, and each provider may have distinct methods and steps to configure this connection. Refer to the respective guide for specific details.

Connecting your Terraform workspace or Stack to the desired branch in your VCS repository is a key step in the VCS-driven workflow. By linking the workspace and the branch, you enable automatic synchronization of runs with code changes.

HCP Terraform provides three main options to specify which changes trigger runs in your repository.

1. When a specific branch changes. This implies long-running branches or merged feature branches to the default branch.
2. Any tags to a repository matching a pattern (per (1)). Branches are irrelevant for this option. A single change in a repository can cause changes in multiple workspaces if customers are mapping a given repo-branch combination to multiple workspaces. This option instructs HCP Terraform to begin new runs only for changes that have a specific tag format. You can choose between various tag format options:
    - Semantic versioning: It matches tags in the popular semantic versioning format. For example, 0.4.2.
    - Version contains a prefix: It matches tags which have an additional prefix before the semantic versioning format. For example, version-0.4.2.
    - Version contains a suffix: It matches tags which have an additional suffix after the semantic versioning format. For example, 0.4.2-alpha.

- Custom Regular Expression: You can define your own regular expression for HCP Terraform to match against tags.

> **Note**
>
> When managing your workspace with the `hashicorp/tfe` provider and triggering runs through matching git tags, remember to include an additional `\` to escape the regular expression pattern.

Tag format	Regex pattern
Semantic Versioning	`^\d+.\d+.\d+$`
Version contains a prefix	`\d+.\d+.\d+$`
Version contains a suffix	`^\d+.\d+.\d+`

3.3 Auto apply

Terraform offers a *auto apply* option, which applies changes from successful plans without prompting for approval (this is irrespective of way the plan triggers - VCS, API and so on). This feature is useful in non-interactive non-production environments to automatically run a `terraform apply` on a successful plan.

Auto apply is a configurable setting per workspace, which means you can control which workspaces run automatically and which need approval. A typical pattern is to allow auto apply for pre-production workspaces, but not for production workspaces where change control is a requirement.

If using Stacks, deployment group orchestration rules govern auto apply behavior, which the deployment configuration codifies. This gives you greater control over which deployments need approval. For example, auto-apply only occurs if the deployment is pre-production *and* where the run plan cites no resources to destroy.

3.4 Branching strategy

Having a well-defined branching strategy is important as it allows for effective management of different stages of development and deployment, providing control over the code changes introduced at each phase. The guidance differs for workspaces and Stacks, due to their different approaches to the promotion of changes through environments.

3.4.1 Branching strategy for workspaces

Deploy each environment or stage using a separate workspace or set of workspaces, and the branching strategy must reflect that mapping. As an overview, here are workspace configuration guidelines that assist you in maintaining a well-organized, scalable, and efficient Terraform workflow:

- Unique repository/branch mapping: The recommended pattern is to assign each workspace to a unique combination of a repository and branch. When changing the associated repository and branch, a Terraform run automatically triggers the corresponding workspace.
- Sub folders within repository: While sub folders can exist within the repository, it is important to ensure that the same workspace manages all resources in the repository. This practice promotes isolation and organization, making it easier to manage and maintain Terraform configurations.
- Branches for related workspaces: Utilize different branches to represent related workspaces. For example, you can have a branch named `main` for production and another branch named `dev` for development. This approach helps to separate and manage configurations for different environments efficiently.
- Pull request (PR) workflow: When a PR opens against a specific workspace branch, the PR must only contain information about that particular workspace. This focused approach allows for clearer review and approval of changes specific to the intended workspace.
- Release workflow: The workflow for releasing changes follows standard development practices. Developers make changes on a feature branch, which is later merged into a long-lived branch like `dev`. Merge changes from the `dev` branch into the `main` workspace, enabling a controlled and streamlined release process.

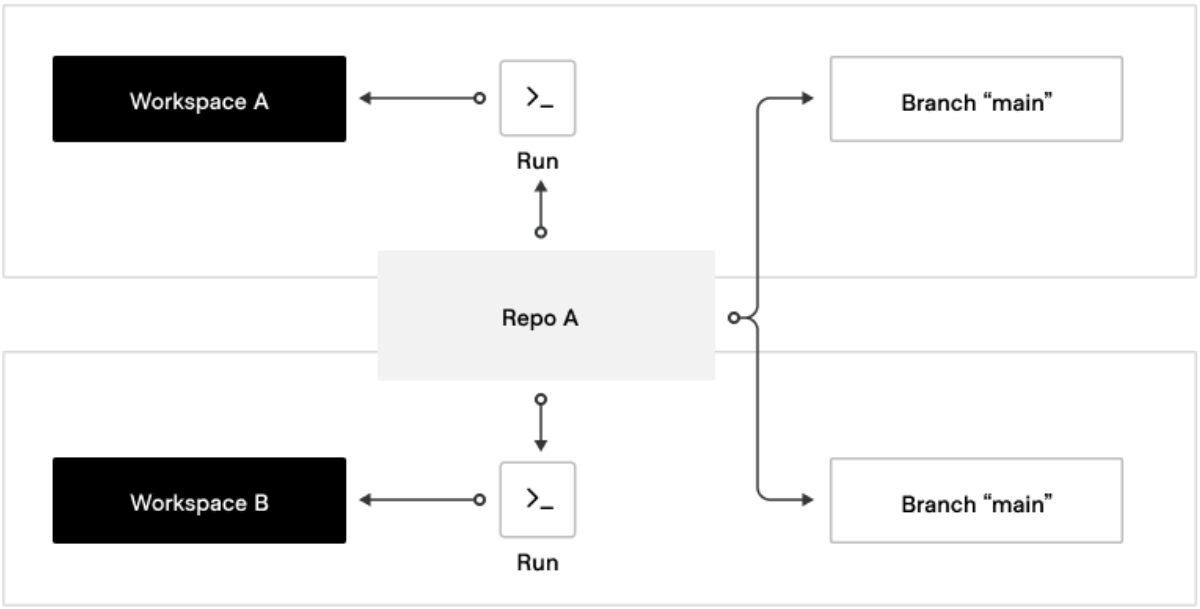


Figure 4: Branching strategy

Recommendation

Consider adopting a branching strategy similar to GitHub Flow, if GitHub is your primary version control tool. This trunking-based model utilizes short-lived branches instead of feature tags. With this approach, you create a branch from the main branch, make your changes (in our Terraform use case, these changes would be within the environment's specified workspace folder) and commit the changes. Then you push a pull request, merge the changes into the main branch, and then delete the branch. The main objective of this model is to have short-lived branches, ensuring that the main branch always reflects the latest features or changes at any given time.

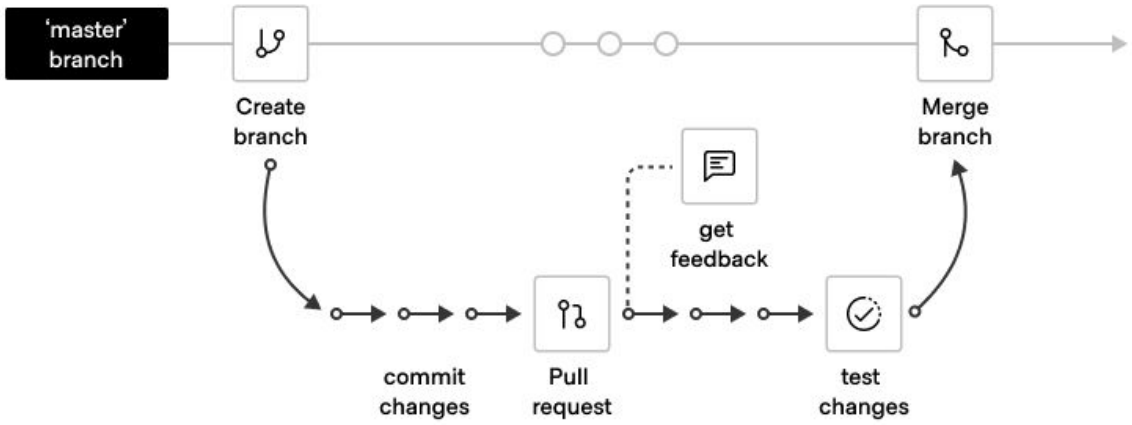


Figure 5: Using short-lived branches

As a general approach, we suggest aligning the `dev`, `stage`, and `production` branches with their respective workspaces for applications. This alignment promotes a clear and structured workflow, enabling smooth transitions between different development stages and deployments.

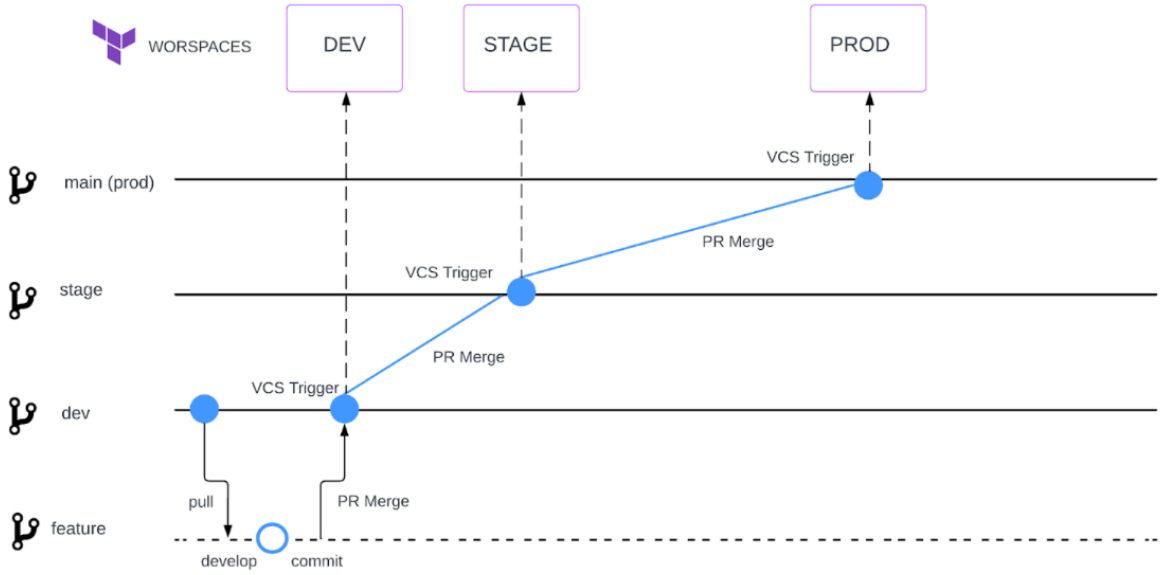


Figure 6: Changes promotion path

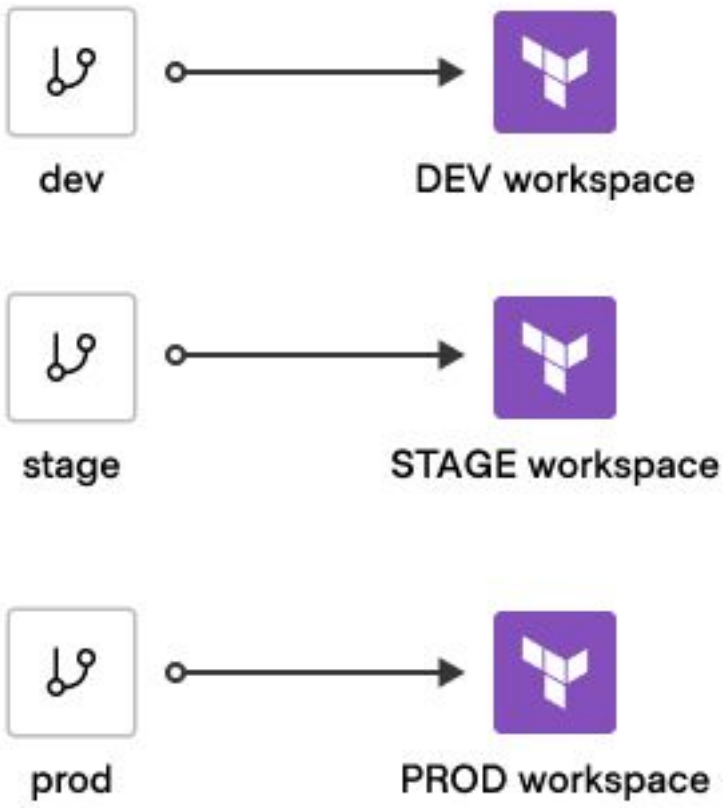


Figure 7: Mapping of branches to workspaces

Here are the steps that align with the GitHub Flow Git Branching Model:

1. Start by creating a repository with three branches - `dev`, `stage`, and `prod`. Commit your initial code in the dev branch.
2. For each feature development, create a separate feature branch off the `dev` branch. For example: `feature/app1-web-tier`.
3. Make your changes and commit them to your branch. Ensure that each commit contains an isolated and complete change for convenient reversion and feedback.
4. Push your changes to your feature branch and create a pull request.
5. Address any comments, issues, tasks, or questions from reviewers during the pull request

review process.

6. The reviewer merges your pull request, incorporating your feature branch into the `dev` branch.
7. The final reviewer performs a squash commits and merge, streamlining the changes into the `dev` branch.
8. Delete the feature branch after merging to maintain only long-lived branches for each environment (dev, stage, and prod).

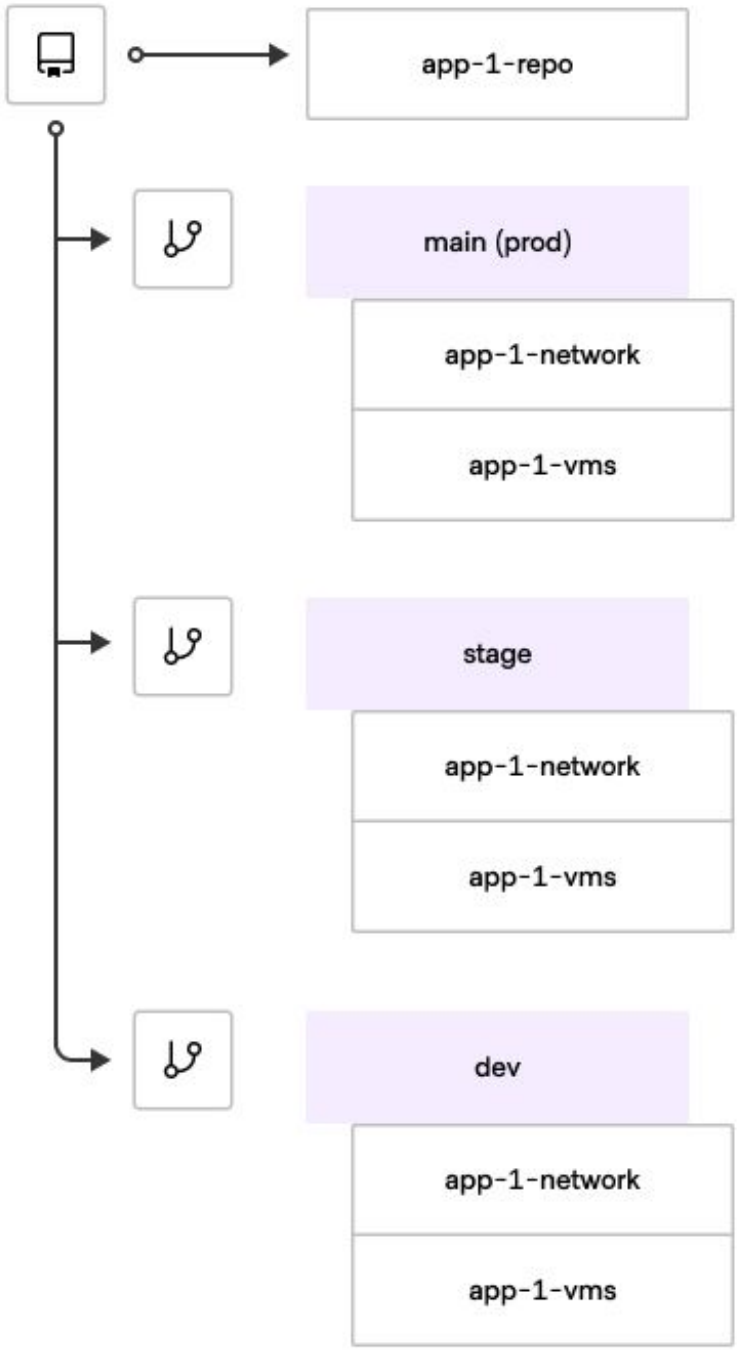


Figure 8: GitHub flow git branching model

Speculative plan

Speculative plans play a crucial role as they allow you to preview and validate changes before merging them into the main branches. This ensures that your infrastructure remains stable and reliable. Here are the steps for the code update workflow, taking advantage of the recommended branching strategy and utilizing speculative plans:

1. Create a *feature branch* from the `dev` branch to work on specific changes.
2. Test and validate the features in the feature branch.
3. Open the PR. This triggers a speculative run.
4. Review, approve and merge the changes into the `dev` branch.
5. Repeat the steps 3 and 4 for `stage` branch and then the `main` production branch.
6. Delete the feature branch to avoid the maintenance of long-lived feature branches.

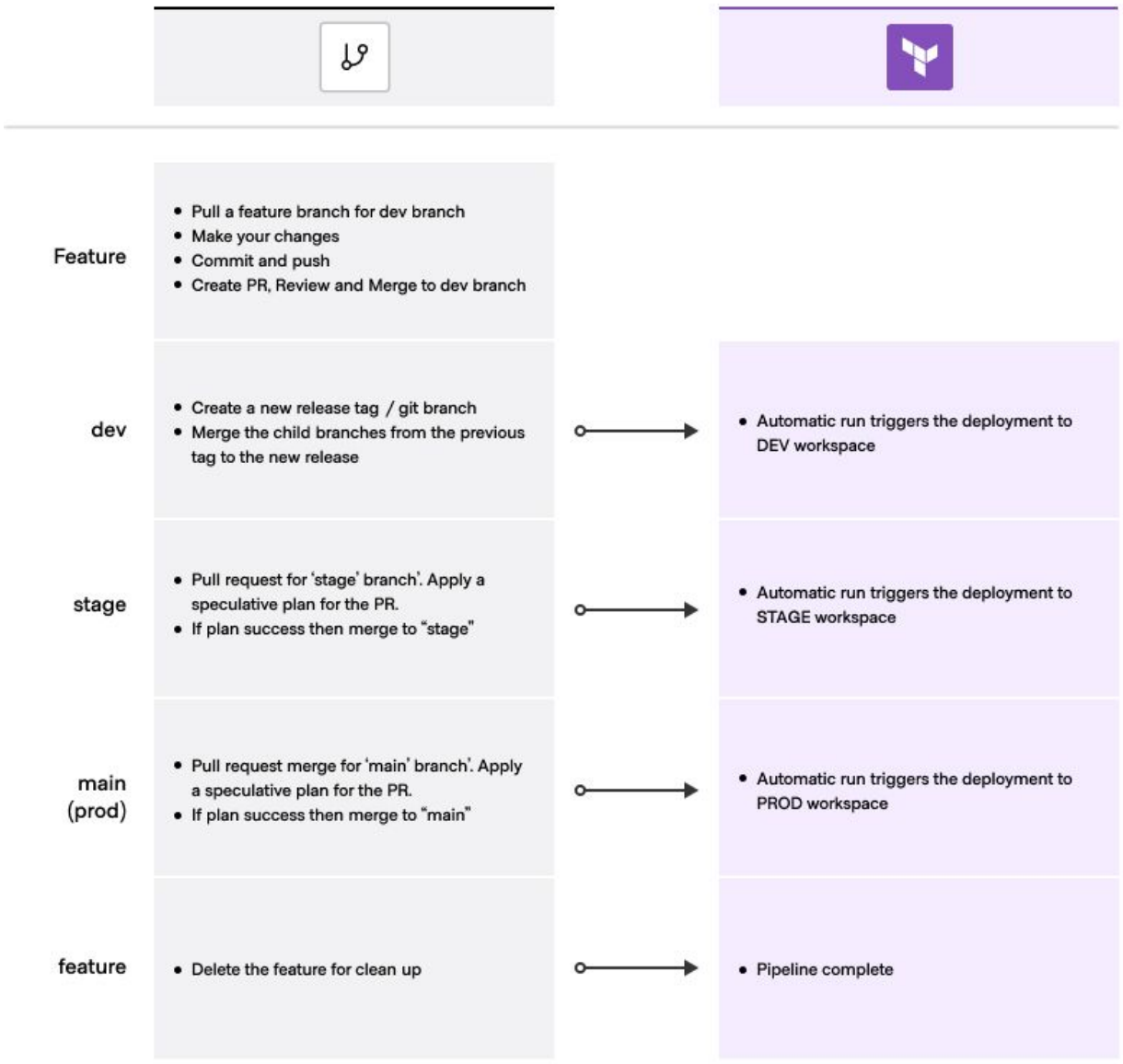


Figure 9: Speculative plans and code deployment

Speculative plans are a valuable feature for reviewing proposed changes. Terraform can automatically run speculative plans for pull requests or merge requests. You can view speculative plans in the UI. Also, if you are using GitHub, Terraform adds a link to the run in the pull request itself, along with an indicator of the run's status. When you update a pull request, Terraform performs new speculative plans and updates the link.

Although any contributor to the repository can see the status indicators for pull request plans, only members of your Terraform organization with permission to read runs for the affected workspaces can click through and view the complete plan output.

When creating a pull request or updating one, Terraform checks whether a speculative plan connected to that repository needs to run in the connected workspace, based on the following rules:

- Only pull requests that originate from within the same repository can trigger speculative plans.
- Pull requests can only trigger runs in workspaces with automatic speculative plans enabled. You can turn off automatic speculative plans in the workspace VCS settings.
- A pull request only triggers speculative plans in workspaces connected to that pull request's destination branch. In order to trigger an apply run, close the PR and merged into the branch, which means statuses are no longer registered.

HCP Terraform does not update the status checks on a pull request with the status of an associated apply. This means that a commit with a successful plan but an apply error still shows the passing commit status from the plan.

3.4.2 Branching strategy for Stacks

Note

At the time of writing, rollout across environments is controlled using auto-apply conditions and manual approvals. More advanced rollout and promotion behaviour will be defined declaratively within the Stack configuration as the feature evolves.

The preceding VCS-backed workspace workflow models application environments as separate workspaces. We recommend controlling promotion across environments through version control, typically using pull/merge requests to move changes from one environment branch to the next. In practice, teams often find it difficult to keep long-lived environment branches fully aligned, and configuration drift is common, increasing the risk of unintended differences across environments.

Stacks use an alternative approach. They represent application infrastructure environments as deployments within the same Stack. As a result, all environments track the same codebase, typically the main branch. When you upload a new configuration version to the Stack, all deployments `plan` in parallel, allowing you to control roll out across environments by manually approving deployment runs in the desired order. You can apply the lowest environment (development) without approval by defining auto apply conditions on the deployment group.

Examples of this concept follow.

- A Stack contains three deployments (dev, staging, prod) that share the same component structure.
- Auto-apply rules within the dev deployment group cause automatic `apply` of the dev environment as long as the plan meets the specified conditions, such as where the change schedule does not include resource destruction.
- Once the dev deployment run is complete and the environment build is successful, manually approve the staging deployment group.
- If encountering a problem at any stage, either in completing the apply or in the health of the resulting infrastructure, cancel the plans in the subsequent deployments. Fix any problems in the code and push a new configuration version and repeat the process.
- Apply and test the staging environment, then arrange for the approval of the change request for the production deployment to complete.

Note

Teams with strict VCS-based controls or audit requirements may still prefer a branch-per-environment strategy. In that case, use separate Stacks for pre-prod and prod environments.

3.5 Additional workflow options

While the recommended VCS-driven workflow offers seamless collaboration and automation through integration with your version control system, it is good to consider alternative approaches that might better suit your organization's specific needs. The VCS-driven workflow provides a convenient way to implement an IaC workflow without requiring custom CI/CD pipelines. It is beneficial to explore other two workflows: the command line-driven and API-driven workflows. These approaches have their advantages and can be valuable additions to your Terraform workflow.

3.5.1 API-driven workflow

When working with HCP Terraform and Terraform Enterprise, the API-driven workflow offers complete control over the IaC process, making it highly compatible with custom CI/CD pipelines utilizing the HCP Terraform and Terraform Enterprise API. If your requirement involves seamless integration with external CI tools, an API-driven workflow can be the most suitable approach, providing flexibility to utilize your current CI tools for tracking state and maintaining continuous linkage between Terraform and your code. As the most advanced approach, this workflow is often adopted by large organizations with complex requirements, either directly or as an evolution from the command line-driven workflow.

The API-driven workflow offers distinct advantages over the recommended VCS approach, including unparalleled flexibility for integration with existing CI/CD pipelines, additional testing capabilities, and full integration with third-party platforms. It also enables centralized code management, retains current CI/CD configurations, and leverages existing programming languages for a more tailored Terraform workflow.

API workflow overview

The high-level API-driven workflow utilizing HCP Terraform and Terraform Enterprise API endpoints includes the following steps:

1. Define variables for organization, workspace, and projects using team or user API tokens.
2. Prepare the configuration file for upload using the configuration version API.
3. Solicit the workspace ID using the configuration version API.
4. Create a configuration version using the configuration version API.
5. Upload the configuration version using the run apply API.

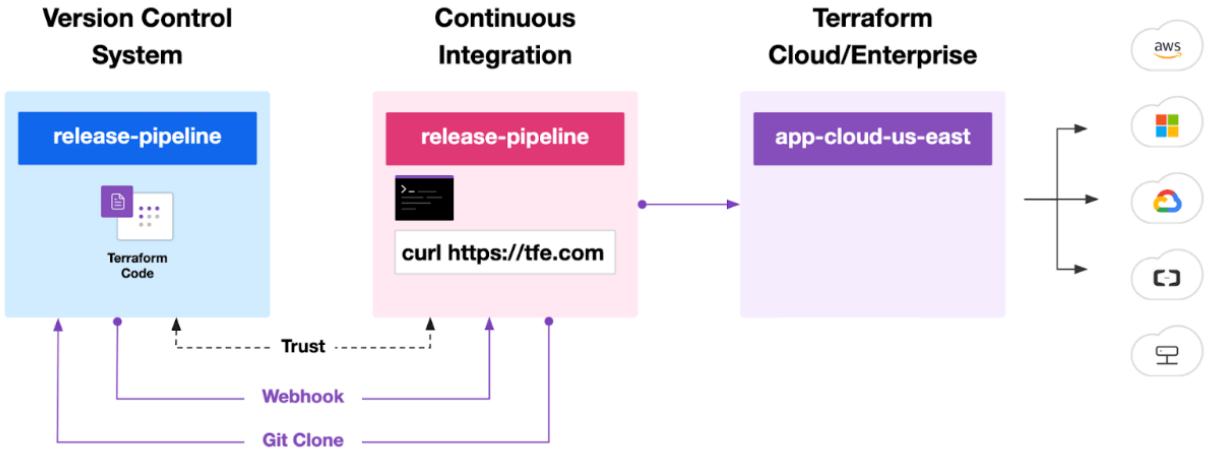


Figure 10: The VCS workflow

3.5.2 Pipeline templates

The API-driven workflow can be further enhanced by leveraging pipeline templates, which bring numerous benefits to integrating HCP Terraform and Terraform Enterprise with various CI/CD platforms. These templates provide clear and predefined examples, making the integration process straightforward and less time-consuming. With a fast-start approach, organizations can streamline their integration process, accelerating the deployment of Terraform in their CI/CD workflows while ensuring consistency across the pipelines. By utilizing existing templates, organizations save effort and resources that is otherwise spent developing custom API-driven workflows. Moreover, these prescriptive templates cater to various CI/CD platforms, ensuring compatibility and smooth integration with existing tools. As an added advantage, the templates are continuously maintained and updated by the Terraform community, allowing organizations to benefit from the latest improvements and best practices, keeping their Terraform workflow up-to-date and efficient.

When integrating Terraform into your existing CI/CD pipelines, consider the convenience and efficiency of leveraging prescriptive pipeline templates. These templates provide explicit examples of how to integrate with HCP Terraform and Terraform Enterprise on various CI/CD platforms, offering a straightforward way to get started without the need to develop templates from scratch. By following these steps, you can incorporate Terraform into your CI/CD workflows.

1. Choose a pipeline template that aligns with your CI/CD platform and meets your specific

integration needs with HCP Terraform and Terraform Enterprise.
2. Tailor the template to suit your organization's requirements by adjusting configurations, adding or removing steps, and integrating it with other tools in your CI/CD pipeline.
3. Integrate the customized template into your CI/CD pipeline and thoroughly test it to ensure it aligns with your desired workflows.

Examples

HashiCorp offers GitHub Actions that integrate with HCP Terraform and Terraform Enterprise APIs, empowering you to create customized CI/CD workflows tailored to your organization's specific needs.

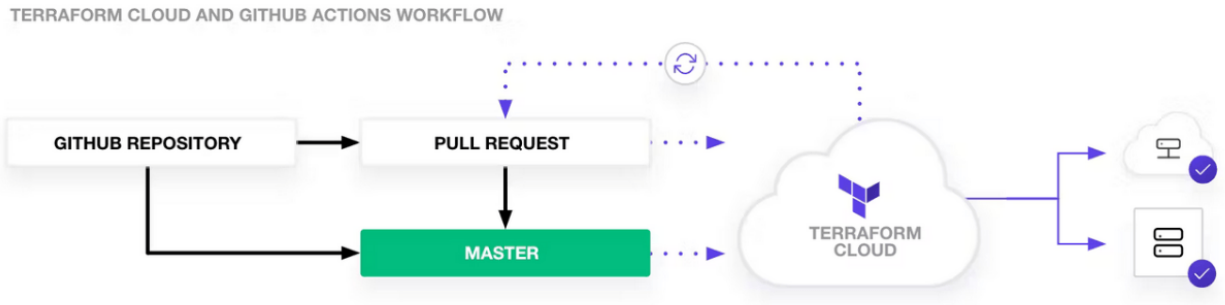


Figure 11: GitHub integration

A typical GitHub Action workflow includes generating a plan for each commit to a pull request branch. This is then reviewed in HCP Terraform. Once you update the VCS branch, the configuration is automatically applied.

Upon configuring the GitHub Action, you can create and merge a pull request to test the workflow. HCP Terraform has built-in support for webhooks to facilitate this generic workflow. Also, by using HashiCorp's Terraform GitHub Actions, you can design a custom workflow with additional steps before or after your Terraform operations, enhancing your automation capabilities.

For additional information and resources, you can refer to the following documentation:

- Announcement
- Tutorial

3.5.3 Command line-driven workflow

The command line-driven workflow, you can transition from using OSS command line commands to HCP Terraform Agents for triggering the build process. This workflow offers the flexibility to iterate on your configuration and work locally, allowing for efficient development and testing. Moreover, it simplifies integration with CI/CD tools, enabling the addition of testing steps and enhancing deployment reliability, streamlining the process compared to the VCS-driven approach. Also, the command line workflow promotes centralized code management, fostering effective collaboration and version control, which is valuable for organizations with complex requirements. Furthermore, it permits convenient local development of workspaces, minimizing the need for frequent code commits and enhancing the overall development experience. For organizations with existing CI/CD configurations, transitioning to the command line workflow can be straightforward, allowing them to retain their current setups and reducing the need for major adjustments. Lastly, the command line workflow provides faster feedback to practitioners, enabling greater flexibility in deploying changes across different environments, making it an attractive alternative to the VCS-driven approach for those who require rapid iterations and deployments.

Workflow overview

This high-level command line-driven workflow includes the following steps:

1. Generate a command line token for use as a CI/CD tool or local user.
2. Create a workspace and update the backend configuration block.
3. Construct CI/CD pipelines to include pre-plan checks like linting, security scanning, and unit testing.
4. Trigger the plan using the terraform plan, apply, or destroy command line command, whether invoked by a human user or a CI/CD script.
5. Configure your CI/CD tool to automatically trigger plans based on branch or directory changes.
6. Set up different pipelines for pull requests, pushes to specific branches, and so on.

After making and validating code changes, merge pull requests to the main branch.

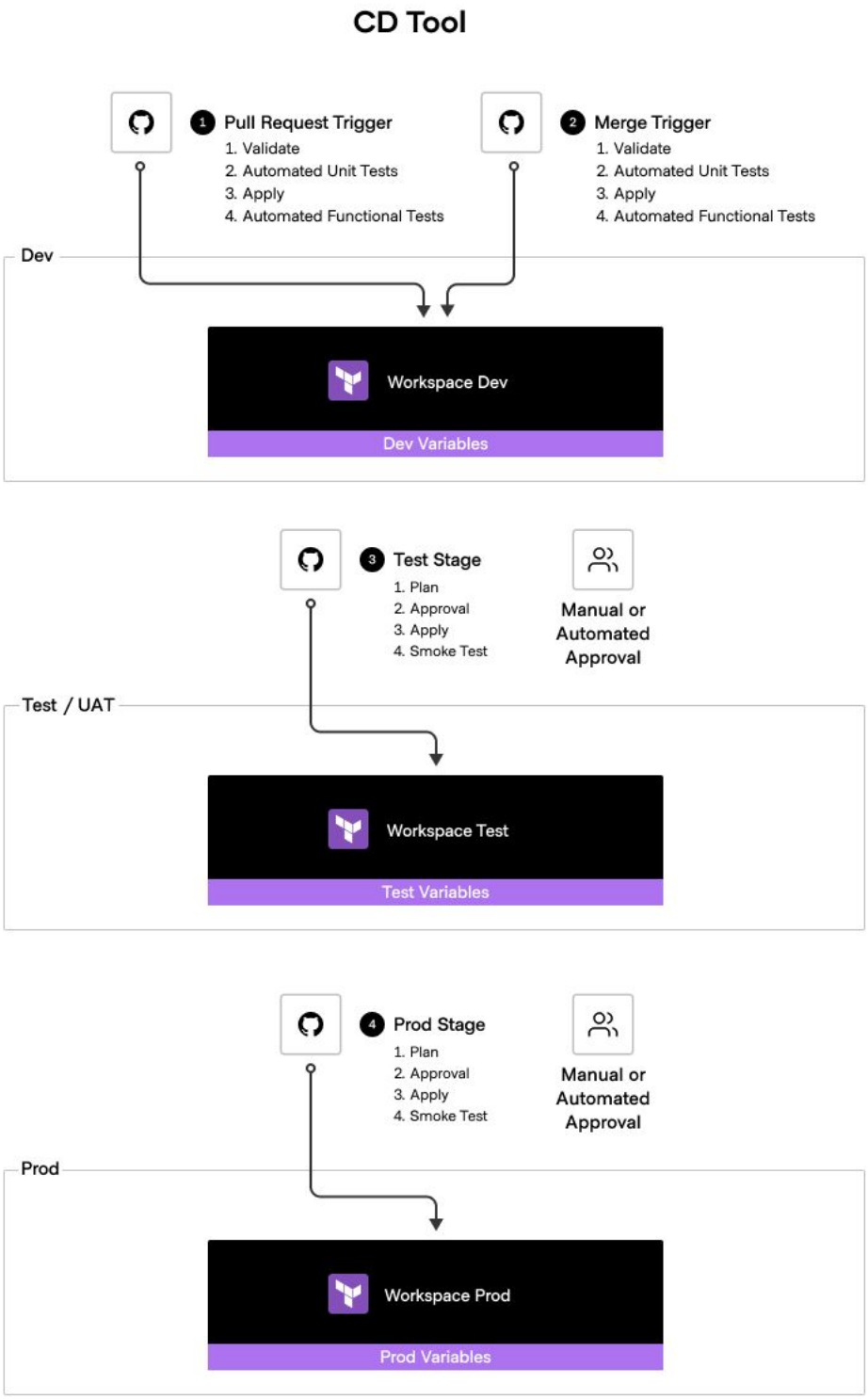


Figure 12: Integration with a CI/CD tool

4 Import existing resources

4.1 Introduction

The Terraform import operation lets you add existing resources to your Terraform state. This is helpful if you have resources created outside of Terraform (using a cloud portal, or another tool) and want to manage them with Terraform. Importing these resources allows Terraform to track and manage their state, simplifying the management of your infrastructure.

As of Terraform 1.5.0, there are two distinct workflows to import existing resources into Terraform state.

- The command line interface (Terraform 0.7.0 or greater)
- The import block (Terraform 1.5.0 or greater)

While straight forward to use, the command line interface method can be time-consuming and carries risk if not handled properly. Terraform 1.5.0 introduced the **`import`** block to address these issues by:

- Allowing you to declare the resources to import in the Terraform configuration.
- Enabling you to import resources in bulk through the use of `for_each`.
- Adding a planning phase to the import process prior to importing the resource.
- Adding a sub-command to generate the configuration for imported resources.

The **`import`** block is the logical progression of the import workflow and the one we recommend. If you are using an earlier version of Terraform and unable to upgrade, see below for guidance. There are also some use cases where it may be better to use the command line interface rather than the import block.

4.2 People and process

We make the following assumptions which may need adapting to your specific organisation. There are two distinct teams who work on infrastructure imports.

The platform team:

- Responsible for setting up and maintaining the Terraform workspaces, projects, organizations, policy sets, cloud authentication, and so on.

- Develops best practices around imports, module usage, which resources to import into modules or root configurations.
- Provides training to the application team on best practices identifying Import use cases.
- Converts repeated configurations into Terraform modules, for use by Waypoint/no code or child modules depending on the responsibilities.

The application team:

- Identifies which resources would be beneficial or non-beneficial to import.
- Works with the platform team to ensure accurate like-for-like transition from non-Terraform-managed to Terraform-managed infrastructure.
- Provides all information to the platform team regarding which resources need which attributes configured or not, and whether they would be reusable as a module.
- Runs the day-to-day Terraform plans/applies through the standardized use of modules or best practices provided by the platform team.

4.3 HashiCorp guidance

We recommend adding `import` blocks in a dedicated `imports.tf` file, similar to defining variables in a `variables.tf`.

4.3.1 What to import

- Resources created by other methods, be it other IaC, other tooling, or manual click ops.
- Resources that cannot be rebuilt with downtime using snapshot, imaging or backup methods.

4.3.2 What not to import

- Dynamic infrastructure where the future changes are outside of Terraform.
- Temporary or experimental resources, or ephemeral workspaces.
- Resources unsupported by providers that would necessitate using `null_resource`.
- Resources managed by teams who do not use Terraform or are not yet ready to use Terraform.
- Resources that can be rebuilt with downtime. It may be easier to create a new configuration and decommission existing resources.

4.3.3 Following a phased approach

Phase 1:

- The application team needs guidance from the platform team on which resources to manage with Terraform, paying close attention to the examples in the *what not to import* section.
- Once you decide on an initial pilot project with a small set of resources, finish this to completion, documenting the process for future teams.
- Gradually expand more resources from the same team, or when the team exhausts its resources, move on to other teams with sufficient knowledge.
- Continuously review and refine the process, making sure the application teams maintain the day 2 and onward responsibilities, and any and all module suggestions.
- The platform team enables features such as drift detection to ensure changes made are within Terraform itself.

Phase 2:

- After completing phase one, you may have noted some common configurations forming.
- Placing common resources together into modules helps standardize Terraform consumption.
- We advise adding some granularity between your modules, accounting for permissions and security. You cannot partially instantiate a module in a configuration file. Splitting into more than one module may add flexibility.
- Using projects, variable sets and Sentinel policy sets may help you in setting out your new configuration files.
- Turning on drift detection and health checks may provide good oversight as to if and when resources are being manually tampered with.

4.3.4 A comparison of the two methods

The following table provides a comparison of the two methods, helping you understand the differences.

Characteristic	Terraform `import` command	Terraform `import` block
How many resources can you import at once?	One resource per command execution.	One or more resources per command execution.
Where is the resource ingested?	Terraform adds the resource to the state file	Terraform first adds the resource to the configuration and then adds it to the state (and marks it as imported).
Is writing resource configuration required?	Yes	Yes, but the process is greatly sped up by the `-generate-config-out` sub-command.
Driven by?	The practitioner (manually).	Configuration and the Terraform workflow.

4.4 Using the Terraform `import` block

Tip

The Import Terraform configuration tutorial also provides example steps for using the import block.

4.4.1 Import block overview

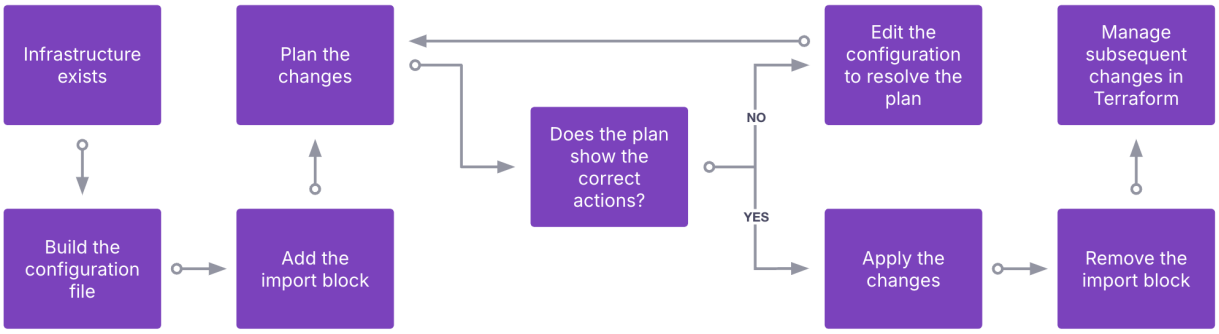


Figure 13: Full workflow

4.4.2 Building a configuration file

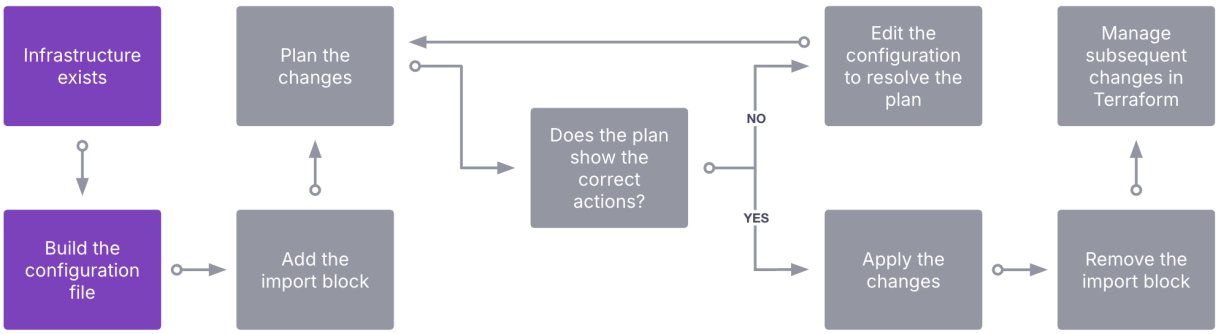


Figure 14: Build configuration

The infrastructure must already exist, and your credentials must allow you to access the resource. You now need to define the scope of your configuration file. Including more resources in your configuration file increases the risk of blast radius and privilege issues. Review best practices for guidance on logical granularity when defining configurations.

You can also use the import block to directly import into a module call. If your configuration uses modules, you can do this in a single step, rather than first importing into a root configuration and later transitioning to a module structure.

4.4.3 Adding the `import` block

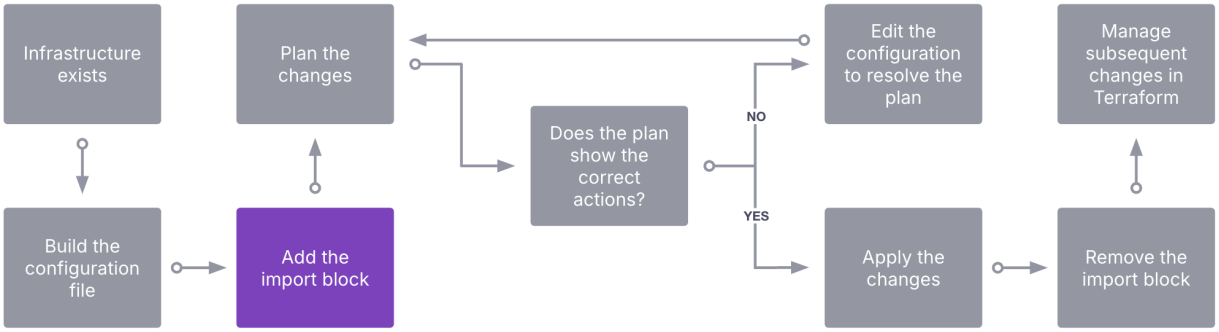


Figure 15: Add import block

The **`import`** block has two required inputs, `to` and `id`. For each Terraform resource, there is a unique ID. In the case of the `aws_iot_thing` resource, this is the name, for the `aws_instance`, it uses the instance ID. Each resource's documentation page lists the required identifier with an example import block.

- `aws_instance` resource documentation
- `aws_iot_thing` resource documentation

If you need to import multiple instances of the same resource, you can use the `for_each` argument within the **`import`** block. At the end of this section there are examples demonstrating how to use `for_each` to import multiple instances of the same resource into root configurations or modules. Note that this approach only applies to resources of the same type. If you need to import multiple different resource types, you need to use separate **`import`** blocks for each type.

4.4.4 Planning changes

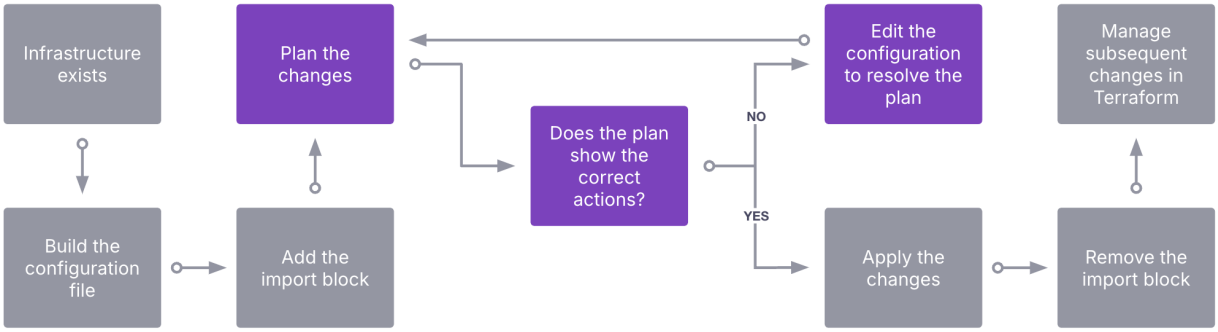


Figure 16: Plan changes

The application team is ready to run the first `terraform plan`. We would expect to see the output only contain import actions. For example:

```
Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.
```

If you expect any changes to add, change, or destroy, review these changes and update your configuration file to reflect them. For example, a configuration change made to a manually created resource which needs an explicit attribute set in Terraform. You repeat this until all you see in the plan is an expected number of imports.

4.4.5 Applying changes

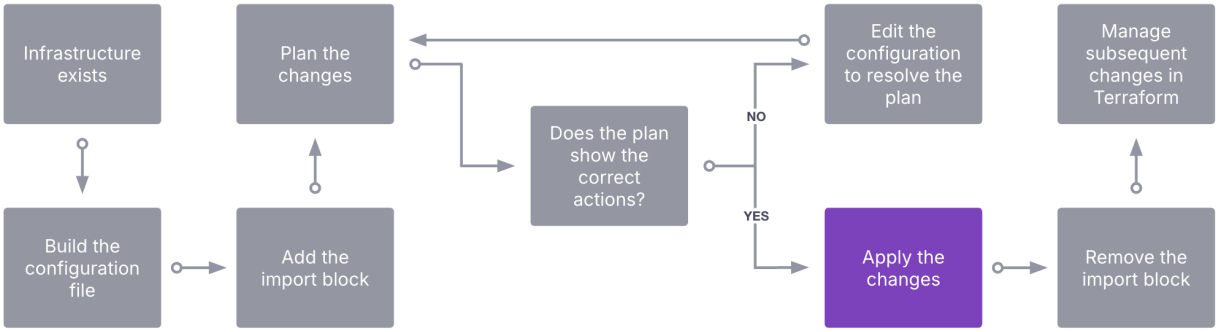


Figure 17: Apply changes

Once the plan shows one or more imports, with zero additions, changes, and destroys you can continue with an apply command.

```
$ terraform apply

aws_iot_thing.bar: Importing... [id=foo]
aws_iot_thing.bar: Refreshing state... [id=foo]

Terraform will perform the following actions:

  # aws_iot_thing.bar will be imported
    resource "aws_iot_thing" "bar" {
        arn               = "arn:aws:iot:eu-west-1:1234567890:thing/foo"
        attributes        = {}
        default_client_id = "foo"
        id                = "foo"
        name              = "foo"
        version           = 1
    }

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iot_thing.bar: Importing... [id=foo]
aws_iot_thing.bar: Import complete [id=foo]
aws_iot_thing.bar: Refreshing state... [id=foo]

Apply complete! Resources: 1 imported, 0 added, 0 changed, 0 destroyed.
```

4.4.6 Final steps

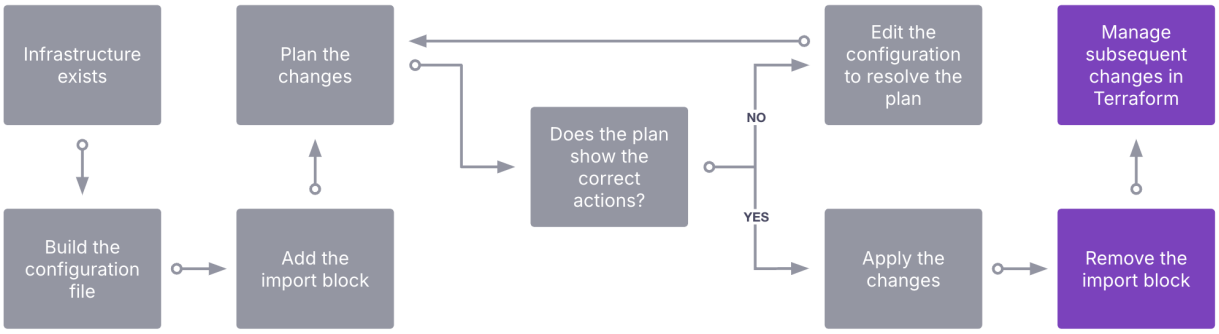


Figure 18: Final steps

With the resources imported, remove the `import` blocks. The import process is complete. Make all subsequent changes in Terraform code rather than manual interventions/other tools. This avoids drift which can be a problem in itself. Enabling drift detection on your workspace can help diagnose teams or processes making changes outside of Terraform.

4.4.7 Import block examples

Import one resource into a base configuration file

```
## main.tf
resource "aws_iot_thing" "bar" {
  name = "foo"
}
```

Create `imports.tf` with the following code block.

```
## imports.tf
import {
  to = aws_iot_thing.bar
  id = "foo"
}
```

Run `terraform plan`.

```
$ terraform plan

aws_iot_thing.bar: Preparing import... [id=foo]
aws_iot_thing.bar: Refreshing state... [id=foo]

Terraform will perform the following actions:

  # aws_iot_thing.bar will be imported
    resource "aws_iot_thing" "bar" {
        arn               = "arn:aws:iot:eu-west-1:1234567890:thing/foo"
        attributes        = {}
        default_client_id = "foo"
        id                = "foo"
        name              = "foo"
        version           = 1
    }

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.
```

Check there are no unexpected outputs to the plan, then run `terraform apply`.

```
$ terraform apply

aws_iot_thing.bar: Importing... [id=foo]
aws_iot_thing.bar: Refreshing state... [id=foo]

Terraform will perform the following actions:

  # aws_iot_thing.bar will be imported
    resource "aws_iot_thing" "bar" {
        arn               = "arn:aws:iot:eu-west-1:1234567890:thing/foo"
        attributes        = {}
        default_client_id = "foo"
        id                = "foo"
        name              = "foo"
        version           = 1
    }

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iot_thing.bar: Importing... [id=foo]
aws_iot_thing.bar: Import complete [id=foo]
aws_iot_thing.bar: Refreshing state... [id=foo]

Apply complete! Resources: 1 imported, 0 added, 0 changed, 0 destroyed.
```

Remove the `imports.tf` file and make all subsequent changes using this Terraform configuration file.

Import one resource into a module

```
## ./main.tf
import {
  to = module.helloworld.aws_iot_thing.bar
  id = "foo"
}

module "helloworld" {
  source = "./modules/iot_thing"
}
```

```
## ./modules/iot_thing
resource "aws_iot_thing" "bar" {
  name = "foo"
}
```

The preceding configuration would produce the following plan output.

```
$ terraform plan

module.helloworld.aws_iot_thing.bar: Preparing import... [id=foo]
module.helloworld.aws_iot_thing.bar: Refreshing state... [id=foo]

Terraform will perform the following actions:

  # module.helloworld.aws_iot_thing.bar will be imported
    resource "aws_iot_thing" "bar" {
        arn               = "arn:aws:iot:us-east-1:1234567890:thing/foo"
        attributes        = {}
        default_client_id = "foo"
        id                = "foo"
        name              = "foo"
        thing_type_name   = null
        version           = 1
    }

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.
```

Import the state with `terraform apply`.

```
$ terraform apply

module.helloworld.aws_iot_thing.bar: Preparing import... [id=foo]
module.helloworld.aws_iot_thing.bar: Refreshing state... [id=foo]

Terraform will perform the following actions:

  # module.helloworld.aws_iot_thing.bar will be imported
    resource "aws_iot_thing" "bar" {
        arn               = "arn:aws:iot:us-east-1:1234567890:thing/foo"
        attributes        = {}
        default_client_id = "foo"
        id                = "foo"
        name              = "foo"
        thing_type_name   = null
        version           = 1
    }

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

module.helloworld.aws_iot_thing.bar: Importing... [id=foo]
module.helloworld.aws_iot_thing.bar: Import complete [id=foo]

Apply complete! Resources: 1 imported, 0 added, 0 changed, 0 destroyed.
```

Import multiple of the same resource into a root configuration

The following configuration assumes you have created three `aws_iot_thing` resources with the names foo1, foo2 and foo3, and you want to import them all into a root configuration.

```
#./main.tf
locals {
  iot_things = {
    iot1 = "foo1"
    iot2 = "foo2"
    iot3 = "foo3"
  }
}

resource "aws_iot_thing" "this" {
  for_each = local.iot_things
  name     = each.value
}

import {
  for_each = local.iot_things
  id       = each.value
  to       = aws_iot_thing.this[each.key]
}
```

The preceding configuration would produce the following plan output.

```
$ terraform plan

aws_iot_thing.this["iot3"]: Preparing import... [id=foo3]
aws_iot_thing.this["iot2"]: Preparing import... [id=foo2]
aws_iot_thing.this["iot1"]: Preparing import... [id=foo1]
aws_iot_thing.this["iot1"]: Refreshing state... [id=foo1]
aws_iot_thing.this["iot2"]: Refreshing state... [id=foo2]
aws_iot_thing.this["iot3"]: Refreshing state... [id=foo3]

Terraform will perform the following actions:

  # aws_iot_thing.this["iot1"] will be imported
    resource "aws_iot_thing" "this" {
        arn               = "arn:aws:iot:us-east-1:458689188825:thing/foo1"
        attributes        = {}
        default_client_id = "foo1"
        id                = "foo1"
        name              = "foo1"
        thing_type_name   = null
        version           = 1
    }

  # aws_iot_thing.this["iot2"] will be imported
    resource "aws_iot_thing" "this" {
        arn               = "arn:aws:iot:us-east-1:458689188825:thing/foo2"
        attributes        = {}
        default_client_id = "foo2"
        id                = "foo2"
        name              = "foo2"
        thing_type_name   = null
        version           = 1
    }

  # aws_iot_thing.this["iot3"] will be imported
    resource "aws_iot_thing" "this" {
        arn               = "arn:aws:iot:us-east-1:458689188825:thing/foo3"
        attributes        = {}
        default_client_id = "foo3"
        id                = "foo3"
        name              = "foo3"
        thing_type_name   = null
        version           = 1
    }

Plan: 3 to import, 0 to add, 0 to change, 0 to destroy.
```

Import the state with `terraform apply`. Remove the import block and make all subsequent changes in this configuration, not through other tools.

Import multiple of the same resource into one module

The following configuration assumes you have created three `aws_iot_thing` resources with the names foo1, foo2 and foo3, and you want to import them all into a module named `helloworld`.

```
## ./main.tf
locals {
 iot_things = [
   {
     name = "foo1"
     id = "bar1"
   },
   {
     name = "foo2"
     id = "bar2"
   },
   {
     name = "foo3"
     id = "bar3"
   }
 ]
}

import {
 for_each = local.iot_things
 id = each.value.name
 to = module.helloworld.aws_iot_thing.this[each.value.id]
}

module "helloworld" {
 source      = "./modules/iot_thing"
 iot_things  = { for thing in local.iot_things : thing.id => thing.name }
}
```

```
## ./modules/iot_thing

resource "aws_iot_thing" "this" {
 for_each = var.iot_things
 name     = each.value
}

variable "iot_things" {
 type = map(string)
}
```

The preceding configuration would produce the following plan output.

```
$ terraform plan

module.helloworld.aws_iot_thing.this["bar3"]: Preparing import... [id=foo3]
module.helloworld.aws_iot_thing.this["bar2"]: Preparing import... [id=foo2]
module.helloworld.aws_iot_thing.this["bar1"]: Preparing import... [id=foo1]
module.helloworld.aws_iot_thing.this["bar3"]: Refreshing state... [id=foo3]
module.helloworld.aws_iot_thing.this["bar2"]: Refreshing state... [id=foo2]
module.helloworld.aws_iot_thing.this["bar1"]: Refreshing state... [id=foo1]

Terraform will perform the following actions:

  # module.helloworld.aws_iot_thing.this["bar1"] will be imported
    resource "aws_iot_thing" "this" {
        arn               = "arn:aws:iot:us-east-1:1234567890:thing/foo1"
        attributes        = {}
        default_client_id = "foo1"
        id                = "foo1"
        name              = "foo1"
        thing_type_name   = null
        version           = 1
    }

  # module.helloworld.aws_iot_thing.this["bar2"] will be imported
    resource "aws_iot_thing" "this" {
        arn               = "arn:aws:iot:us-east-1:1234567890:thing/foo2"
        attributes        = {}
        default_client_id = "foo2"
        id                = "foo2"
        name              = "foo2"
        thing_type_name   = null
        version           = 1
    }

  # module.helloworld.aws_iot_thing.this["bar3"] will be imported
    resource "aws_iot_thing" "this" {
        arn               = "arn:aws:iot:us-east-1:1234567890:thing/foo3"
        attributes        = {}
        default_client_id = "foo3"
        id                = "foo3"
        name              = "foo3"
        thing_type_name   = null
        version           = 1
    }

Plan: 3 to import, 0 to add, 0 to change, 0 to destroy.
```

Import the state with `terraform apply`. Remove the import block and make all subsequent changes in this configuration, not through other tools.

Import multiple of the same resource into modules with `for_each`

The **`import`** block allows the use of `for_each` if the resource types are identical.

```
## ./main.tf

locals {
 iot_things = {
   iot1 = "foo1"
   iot2 = "foo2"
   iot3 = "foo3"
 }
}

module "helloworld" {
 for_each = local.iot_things
 source   = "./modules/iot_thing"

 name = each.value
}
```

```
## ./imports.tf

import {
 for_each = local.iot_things
 id       = each.value
 to       = module.helloworld[each.key].aws_iot_thing.this
}
```

```
## ./modules/iot_thing

variable "name" {
 type = string
}

resource "aws_iot_thing" "this" {
 name = var.name
}
```

Run `terraform plan` to see the planned changes. Import the state with `terraform apply`. Re-

move the `import` block and make all subsequent changes in this configuration, not through other tools.

Import different resources into the same state file

Different resources require the use of different `import` blocks. In these different `import` blocks you can use all of the techniques shown. When importing different resources, review our workspaces best practices to learn how to scope differing resources in the same configuration.

4.5 Using the Terraform import command

The `terraform import` command introduced in Terraform 0.7 adds existing resources to Terraform state for ongoing management. The process has two main steps:

- Import the resource to state.
- Update the Terraform configuration file to define the newly imported resource.

The `terraform import` command works by passing the resource address and resource ID as command line arguments. Once the command has completed, the Terraform state includes the imported resource.

The second step is to update the Terraform configuration to match the new state. Add the resource and any known attribute values to the configuration file and use `terraform plan` to identify any missing resource attribute values. Use the provider's documentation to help this step. Add those attribute values to the resource block and repeat the planning process until the Terraform plan command proposes no changes. This means the resource definition in the code matches the resource state.

It is crucial that `terraform apply` is not run on the configuration until the resource definition matches the state. Otherwise Terraform might reconfigure the resource in-place, or delete the resource and re-create it based on the code. Both unwanted outcomes.

4.5.1 Terraform import overview

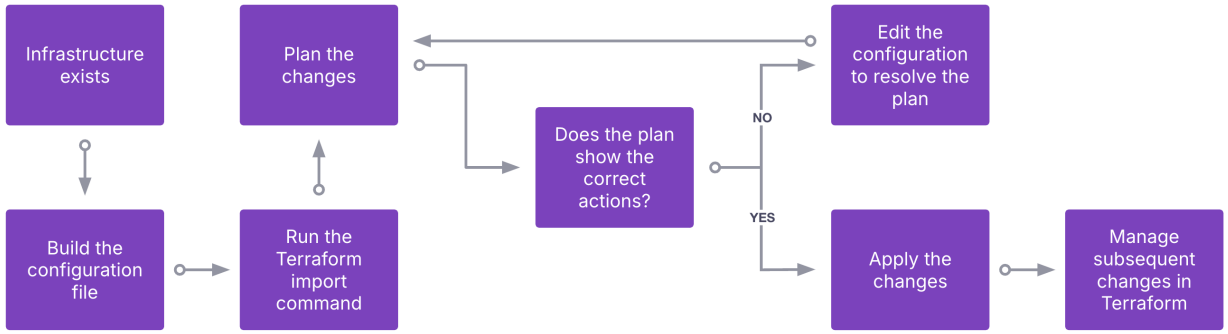


Figure 19: Terraform command-line import process

4.5.2 Import command examples

Import an AWS EC2 instance

Identify the EC2 instance to import and note the instance ID.

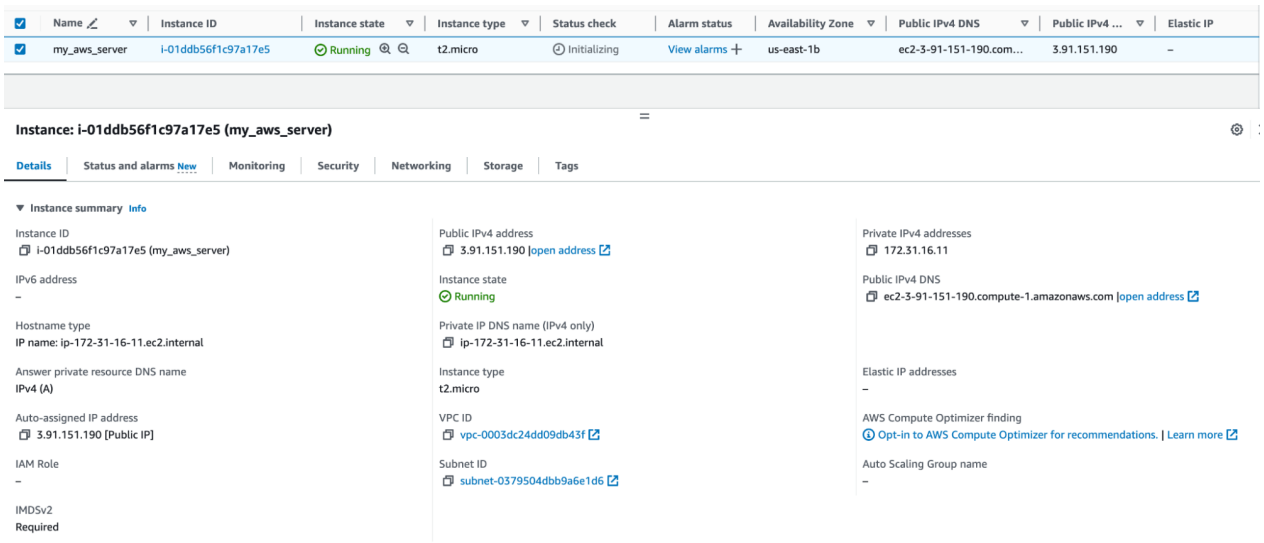


Figure 20: AWS web UI instance list

Write a Terraform configuration file with a resource block for the imported EC2 instance.

```
provider "aws" {

}

resource "aws_instance" "example" {
    ami           = "ami-0c101f26f147fa7fd"
    instance_type = "t3.micro"

    tags = {
        Name = "HelloWorld"
    }
}
```

Run the **`import`** command to add the EC2 instance into Terraform state.

```
terraform import aws_instance.example i-01ddb56f1c97a17e5
      aws_instance.example: Importing from ID "i-01ddb56f1c97a17e5"...
      aws_instance.example: Import prepared!
        Prepared aws_instance for import
      aws_instance.example: Refreshing state... [id=i-01ddb56f1c97a17e5]

Import successful!
```

The command output shows the imported resources. Manage imported resources with Terraform from this point on.

This example shows there are many resource attributes to account for before the EC2 instance is fully managed Terraform. There are more complex cases, where the attributes to configure are more complicated or more numerous than `ami`, `instance_type`, and `tags`.

5 Private Registry

> **Note**
>
> Private registry publishing and curation are configured and managed by platform operators — see the Administration Guide: Private registry management.

5.1 Discovering Terraform Modules and Providers

Whether novice or expert, your Terraform practitioners benefit from an easy-to-use interface to the private registry instance. The discovery of Terraform modules and providers is done from the TFC interface by selecting the **Registry** entry in the left panel.

The web interface is designed to speed discovery. It features tabs to select either the "Modules" or the "Providers" view.

The "Modules" view features a flexible filter that allows users to select modules by provider, source (from the public or private registry), and publishing mode (tags-based or branch-based). The list can also be restricted to show only "no-code ready" modules.

The web interface allows quick access to a number of useful information about a module, such as:

- The module's README documentation
- Input variables
- Output variables
- Dependencies
- Resource types created
- Configuration snippet (which can be copied into the clipboard in a single click)
- Module download statistics
- The Terraform Configuration Designer

The "Providers" view is simpler and provides a paginated list of available providers.

The web interface allows quick access to a number of useful information about a module, such as:

- Module documentation
- Provider download statistics
- Source VCS repository
- Configuration snippet (which can be copied into the clipboard in a single click)

5.2 Consuming Terraform Modules or Providers

Consuming Terraform modules or providers is relatively simple:

- Find the relevant module or providers in the private registry.
- Use them in your Terraform configuration

As a user of Terraform modules and providers, you should use version constraints because they are an easy way to avoid breaking changes that could happen if you always use the latest version. But you should also:

- Monitor the modules and providers you are using for deprecation notices.
- Set aside time to upgrade to a newer version of the module or provider before its deprecation is effective.
- Volunteer applicable feedback and lessons learned to the module or provider author(s) so that they can be improved for your use cases.

6 Policy as Code - Sentinel

Note

Sentinel policies and policy sets are authored and managed by platform operators — see the Administration Guide: Policy as code and Sentinel management.

6.1 Explanation of the policy enforcement workflow

The policy enforcement workflow with HashiCorp Sentinel is a structured process.

- Organizations begin by defining policies that articulate the rules governing their infrastructure.
    - Customers are advised to render their governance and compliance policies in such a way so that Sentinel policies are then authored to satisfy each.
    - Doing so will enable your business to operationalize governance and controls through automated means.
    - Typical policies customers start with include governance of maintenance windows protecting software components from adverse change at the wrong time, and enforcement of metadata tagging of cloud resources, and infrastructure-as-code style such as ensuring Terraform module versions are pinned and usable only from your organization's private registry.

- The next step is to code the policy using the Sentinel language. See below for more on how code repositories are structured.
    - Policies are arranged in sets.
    - Policy sets are scoped to an entire organization or to one or more projects and/or workspaces in your HCP Terraform organization.
    - Scoping policy sets at the project level allows policy enforcement to be integrated into self-service workflows, which typically create new workspaces on the fly then run them immediately. Any new self-service workspace will inherit the policy sets that are scoped to the parent project.
- Sentinel is configured to operate according to the policies in the policy sets that are scoped to the running workspace.
    - Depending on the enforcement level (advisory, soft- or hard-mandatory) of the policy, Sentinel will either allow or deny the apply phase of the workspace run.

7 Run Tasks

> **Note**
>
> Run tasks are integrated and managed by platform operators — see the Administration Guide: Run tasks management.

7.1 Overall flow for Run Tasks within a run

Run tasks are invoked before and after the plan and apply stages of a Terraform run. The information that is available at each stage is presented to the run task when invoked. At the pre-plan stage, the information available will mainly be the code and other attributes; at post-plan and later stages, more information associated with the results of the plan will be available.

Selecting the stage to associate a run task to will vary for a number of factors. The pre-plan stage would be used when the run task is going to examine the code and make a determination if entering the plan stage should even be allowed. The other stages primarily depend on how far you want a plan to go before being stopped by a failure result of the run task and what states of provisioning you are able to accept.

For example, a post-plan or pre-apply run task will examine the plan and determine whether an apply should be allowed. A post-apply would be used mainly to test and possibly gather/store information related to the infrastructure that has been provisioned. Check the documentation for the run task being configured if there are particular stages required or suggested to satisfy its requirements.

Multiple run tasks can be defined at a single stage. They will execute in sequence and in no particular order that could be depended on (do not assume any dependencies between run tasks). Once all run tasks complete for a stage, the continuation of the run is determined by the most restrictive enforcement level in each associated run task.

For example, if a mandatory task fails and an advisory task succeeds, the run fails. If an advisory task fails, but a mandatory task succeeds, the run succeeds and proceeds to the next stage. Regardless of the exit status of the run tasks, HCP Terraform displays the status and any related messages are recorded and appear in the run results.

7.2 Available Run Tasks

The common implementation of run tasks will be to implement an existing run task that integrates with a specific complementary product, usually provided and supported by a HashiCorp technology partner that provides that product. These are published in the public Terraform Registry where you can then view the most up-to-date list of available run tasks with links to the partner's usage documentation. Other vendors or community members may offer run tasks for specific integrations, but they will not be published to the public Terraform Registry, and should be handled as any other open source software.

In addition to using existing technology partner integrations, HashiCorp Terraform customers can build their own custom run task integrations. The process of developing custom run tasks is discussed in a separate topic in this documentation set.

8 HCP Terraform Operator for Kubernetes

The HCP Terraform Operator leverages the CNCF operator framework to enable the management of HCP Terraform resources through Kubernetes Custom Resources.

This operator introduces several Custom Resources tailored for the seamless management of HCP Terraform resources:

- **AgentPool**: This resource manages HCP Terraform Agent Pools and Agent Tokens. It also supports on-demand scaling operations for HCP Terraform agents.
- **Module**: Facilitates the implementation of API-driven Run Workflows, streamlining the execution of Terraform configurations.
- **Project**: Oversees the management of HCP Terraform Projects, allowing for organized and efficient project handling within the Terraform ecosystem.
- **Workspace**: Handles the management of HCP Terraform Workspaces, providing a structured environment for resource provisioning and state management.

8.0.1 HCP Terraform Operator Use Cases

The HCP Terraform Operator has two main use cases that we'll describe in the rest of the section:

1. Auto-scaling agent pools for HCP Terraform
2. Self-service infrastructure via Kubernetes native consumption

8.0.2 Auto-scaling agent pools for HCP Terraform

One of the primary use cases of the Terraform Operator is to provide platform teams with the ability to deploy and autoscale HCP Terraform agent pools on Kubernetes. The operator manages the lifecycle of agent deployment and registration via the AgentPool custom resource and can optionally monitor workspace queues to trigger autoscaling based on defined min and max replicas.

You may increase the maximum number of agents up to the value defined in `autoscaling.maxReplicas` or the licensed limit, depending on which limit is reached first. If there are no pending runs, the HCP Terraform Operator will reduce the number of agents to the specified value in `autoscaling.minReplicas` within the timeframe of `autoscaling.cooldownPeriodSeconds`. This mechanism allows you to only deploy additional agents when needed, and control costs.

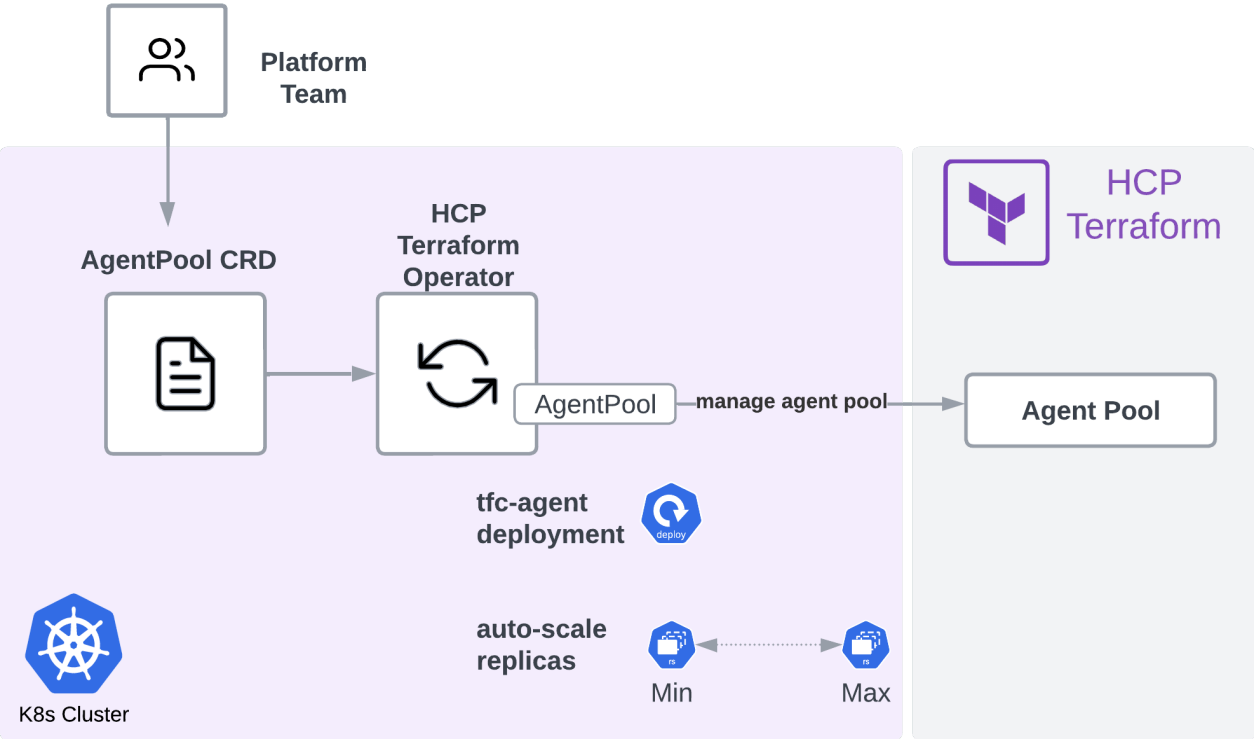


Figure 21: HCP Terraform Operator Agent Pool Scaling

Egress considerations

The network egress requirements for the HCP Terraform agent apply to agents deployed using the HCP Terraform Operator for Kubernetes. These requirements cover the following areas:

- Provider endpoint connectivity
- Access to the Terraform registry
- Access to Terraform releases

See the following HCP Terraform agent documentation sections for details on the agent's connectivity requirements:

- HCP Terraform agent connectivity requirements with HCP Terraform
- HCP Terraform agent connectivity requirements with Terraform Enterprise

Recommendations

HashiCorp recommends:

1. Using a dedicated Kubernetes cluster or logical node separation to host HCP Terraform agents
2. Use Kubernetes cluster autoscaling where available to manage cluster capacity dynamically. This recommendation is particularly applicable if you are using autoscaling agent pools and there is a high variance between peak and off-peak HCP Terraform run concurrency.

- Considering baseline run concurrency for health checks (drift detection and continuous validation) to set the autoscaling `minReplicas` parameter.
- Scale test your Kubernetes cluster up to your maximum intended concurrency and ensure sufficient capacity and dynamic scaling is handled under load.

Sizing autoscaling AgentPools

For the HCP Terraform Kubernetes operator agent pools, establishing the right minimum and maximum replicas hinges on balancing your infrastructure's run concurrency needs and cost-efficiency requirements. The maximum replica count should be determined by predicting peak-run concurrency demand and any constraints imposed by your HCP Terraform tier. Your Kubernetes cluster should be scaled-tested to ensure the peak load requirement of all replicas actively executing is handled

AgentPool CRD

For detailed AgentPool CRD documentation see link below. https://github.com/hashicorp/terraform-cloud-operator/blob/main/docs/agentpool.md

8.0.3 Self-Service infrastructure via Kubernetes native consumption

The HCP Terraform Operator for Kubernetes lets application developers define infrastructure configuration using Kubernetes configuration files. It delegates the reconciliation phase to HCP Terraform, which excels in infrastructure management. This frees developers from needing to learn the HashiCorp Configuration Language (HCL) for managing infrastructure tasks.

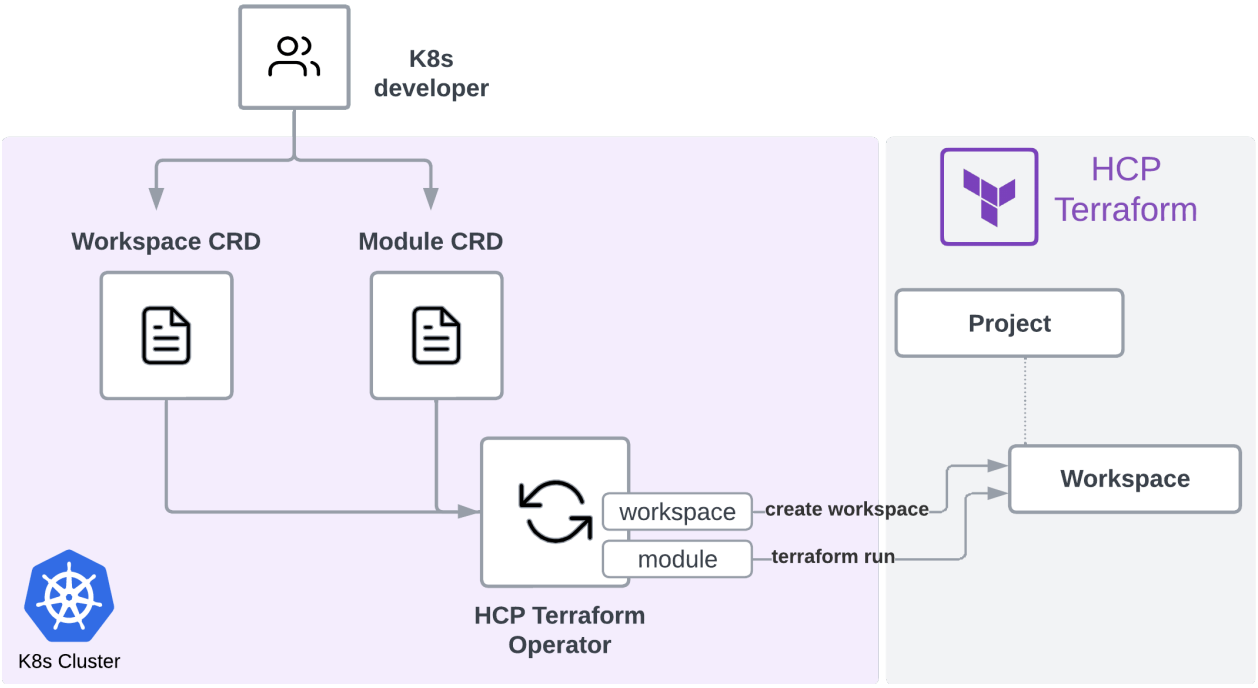


Figure 22: Kubernettes Consumption

The diagram above illustrates the consumption workflow for a Kubernetes developer: The workspace CRD allows for workspace management and the module CRD allows for Terraform run execution control. Commonly, these Kubernetes resources will be consumed through templates using continuous delivery tools such as ArgoCD, which support GitOps practices.

The workspace controller allows for self-service consumption of a workspace and creation within a project. This allows Kubernetes developers to self-serve workspaces within the guardrails of a HCP Terraform project. Through inheritance, this provides a consistent approach for key infrastructure security controls for cloud credentials and policy as code even though the consumption is exposed via Kubernetes.

The module controller provides run execution against a workspace using an API-driven workflow and a Terraform module source. This allows for the separation of producer and consumer such that the consumers within Kubernetes do not need to know Hashicorp Configuration Language (HCL) and can consume approved infrastructure patterns via Kubernetes CRDs and YAML. Modules can be sourced from the HCP Terraform private registry and the module CRD allows run execution control via API-driven workflow.

Recommendations

HashiCorp recommends:

1. Using a single tenant Kubernetes cluster when providing self-service infrastructure with the HCP Terraform Operator.
2. Deploying a single HCP Terraform Operator per Kubernetes cluster and HCP Terraform platform team.
3. Using unique (dedicated) team tokens per HCP Terraform Operator and Kubernetes cluster combination.
4. Leveraging the autoscaling function of Kubernetes cluster, if that function is available (managed services such as EKS Fargate, AKS auto scaler, GKE autopilot do include that function).

Note

Though it's preferable adhering to a single tenant approach, multiple HCP Terraform Operators can be deployed to the same cluster and filters utilized to control namespace access.

When using the HCP Terraform Operator to enable self-service infrastructure for Kubernetes developers the recommended approach is to leverage a dedicated cluster per HCP Terraform project; this approach simplifies scaling and security isolation and allows for team-based cost allocation. Auto-scaling clusters are preferable to simplify capacity management.

8.0.4 Installing the Operator

The operator is installed in your Kubernetes cluster using a Helm chart.

You can install the HCP Terraform operator using Helm CLI directly or using the Terraform helm provider.

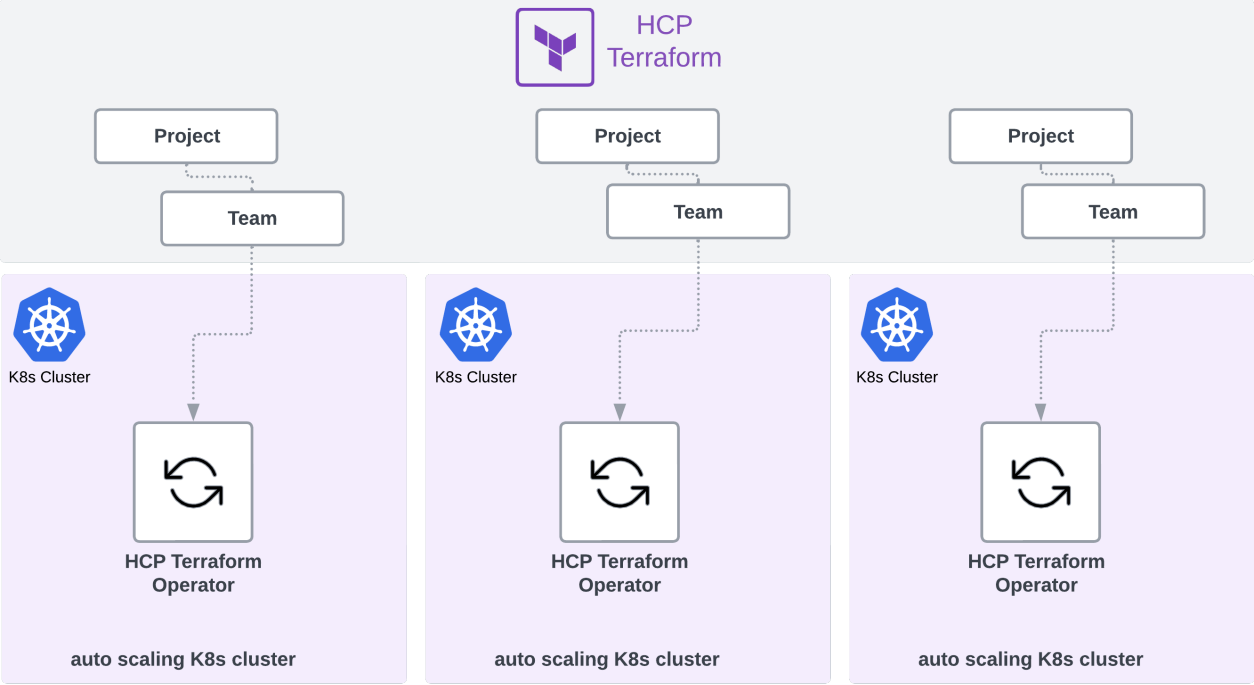


Figure 23: K8 Operator Installation

Instructions on how to install the operator can be found on our documentation site: Set up the HCP Terraform Operator for Kubernetes.

8.0.5 Security considerations

Token requirements

The HCP Terraform Operator resources utilize a Kubernetes secret to reference a HCP Terraform token (see Vault Secrets Operator with HCP Terraform Operator to improve your security posture).

Example secret reference AgentPool CRD

A secret is referenced using secretKeyRef in the AgentPool CRD

```
token:
   secretKeyRef:
     name: tfc-operator
     key: token
```

AgentPool Controller

The AgentPool requires a HCP Terraform token to be generated, this can be referenced using a Kubernetes secret.

Permissions required

Functionality required	Team Permissions
Create and manage agent pools	Manage agent pools (organization-level permission)

Module and Workspace Controller

The module and workspace controllers are targeted at an application consumer or developer persona for self-service infrastructure. To enable self-service with controls within a project the recommended approach is for platform teams to create a project and team access for each team as part of the onboarding process for an application team, this project can be managed by the platform teams either using the TFE provider or using the project controller via the HCP Terraform Operator.

Consumers can create their workspaces within their project by referencing the project id from within the workspace CRD. Specifying a project id enables self-service with the separation of teams via HCP Terraform projects, additionally, this also enables inherited configuration for credentials and policy.

Functionality required	Team Permissions
Create and manage workspaces within a project	Team Token Project Team Access Maintain - Full control of everything in the project, but not the project itself

Project Controller

The project controller provides the ability to deploy and manage HCP Terraform projects. As this is typically a platform team function and requires higher access, if using the controller this will be by a separate HCP Terraform operator that is dedicated to the platform team.

Functionality required	Team Permissions
Create and manage projects and team access	Owners Team Token

Vault Secrets Operator with HCP Terraform Operator

The Vault Secrets Operator allows for secrets to be synced between Vault and the Kubernetes secrets into a specified namespace. Within that namespace, applications have access to the secrets. See https://github.com/hashicorp/vault-secrets-operator

The HCP Terraform operator reads in a HCP Terraform token via the CRD from a referenced Kubernetes secret.

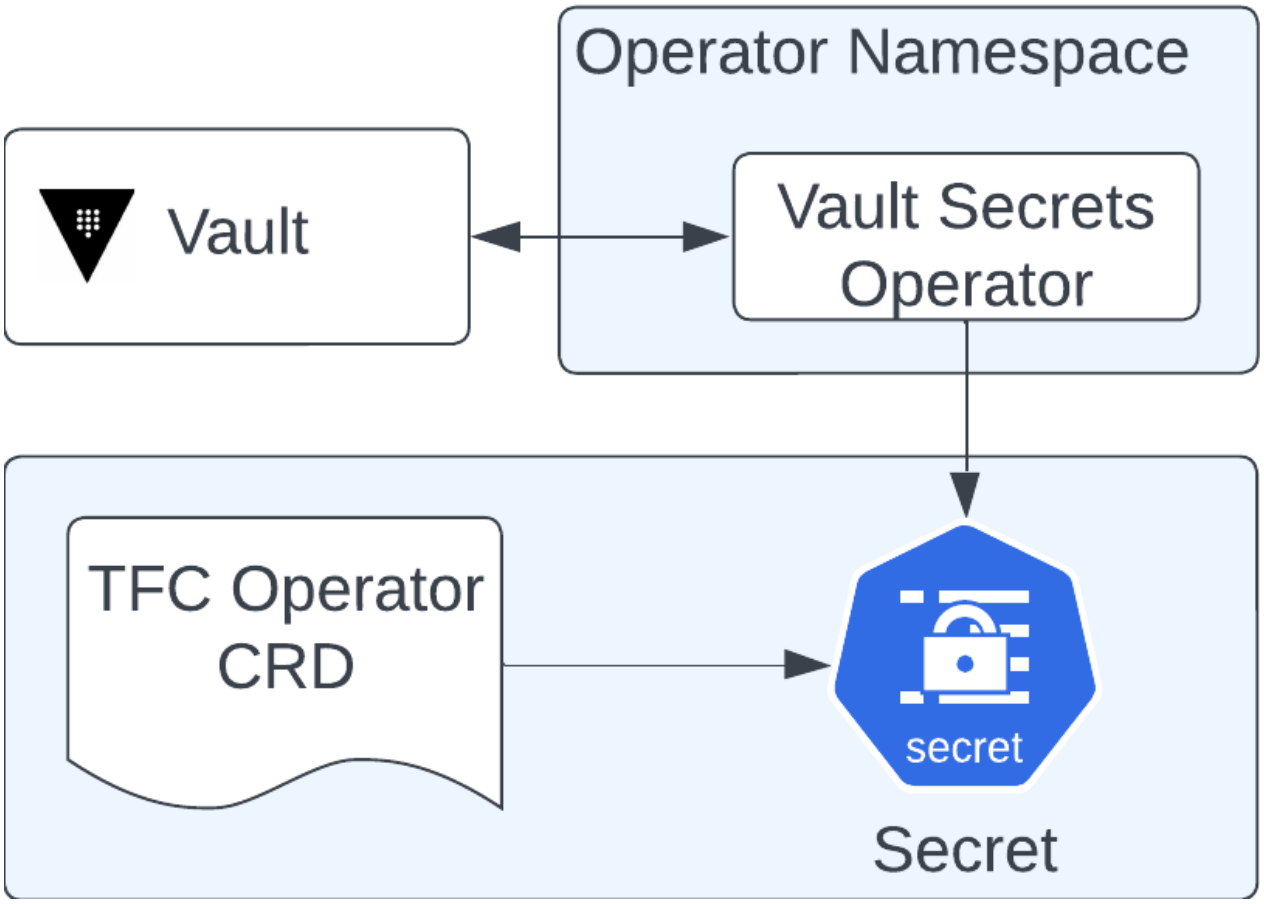


Figure 24: Secrets w/Vault

8.0.6 Observability

The Operator exposes metrics in the Prometheus format for each controller. They are available at the standard `/metrics` path over the HTTPS port 8443. The Operator exposes all metrics provided by the controller-runtime by default. The full list you can find on the Kubebuilder documentation.

HCP Terraform Agent Metrics can be collected using OpenTelemetry. When using the HCP Terraform Operator the environment key can be utilized to set TFC_AGENT_OTLP_ADDRESS variable (see https://developer.hashicorp.com/terraform/cloud-docs/agents/telemetry).

9 Continuous validation

HCP Terraform can perform automatic health assessments in a workspace to assess whether its real infrastructure matches the requirements defined in its Terraform configuration. Health assessments include the following types of evaluations:

- **Drift Detection:** Think of your Terraform configuration as the blueprint for your infrastructure. Drift detection automatically spots if the real thing doesn't match the plan anymore. This helps you fix problems before they break things.
- **Continuous validation:** Terraform doesn't just build your infrastructure – it enforces your standards over time. Set up rules for security, cost, or anything important, and Continuous Validation checks that they're always being met. For more details on Continuous Validation, refer to Docs for more details on Continuous Validation.

Note: Health assessments are available in HCP Terraform Plus. Refer to HCP Terraform pricing for details. In Terraform Enterprise, health assessments are part of the *Governance & Policy* module.

Before reviewing the best practices please review the official documentation for Health Assessments in HCP Terraform here.

9.1 Drift Detection

Drift detection in HCP Terraform is designed to ensure the integrity and consistency of your infrastructure. This feature identifies when the actual state of your infrastructure deviates from the expected state defined in your Terraform configurations, a scenario commonly referred to as *configuration drift*.

It's important to understand the differences between configuration drift and state drift. Configuration drift is a change in the configuration of an application or infrastructure. State drift is a configuration state where a resource's actual configuration differs from its expected configuration.

- Configuration drift is a change in the configuration of an application or infrastructure. This occurs when changes are made directly to the infrastructure outside of Terraform's managed processes, leading to discrepancies between the live state and the code-defined desired state. Common causes include manual interventions, emergency fixes, or changes applied through other automation tools.

- Configuration drift differs from state drift. Drift detection does *not detect* state drift. State drift occurs when external changes affecting remote objects do not invalidate your infrastructure configuration.

Limitations

HCP Terraform's drift detection is a powerful tool, but it is important to understand its limitations. While it excels at identifying configuration drift for resources it actively manages, there are certain limitations to keep in mind:

- **Unmanaged attributes:** Drift detection focuses on attributes explicitly managed by Terraform. Changes made directly to resource attributes outside of Terraform's control will not be flagged. For example, if you manually modify a security setting on a virtual machine instance, drift detection wouldn't detect this change.
- **External additions:** HCP Terraform can't detect resources that are added to your infrastructure entirely outside of Terraform's purview. For instance, if you manually create new IAM users within your cloud environment, drift detection wouldn't identify these additions.

Prescriptive Guidance

With Organization owner permissions you can enforce health assessments for all workspaces or allow the setting to be controlled at the individual workspace level. Enforcing health assessments at an organization level overrides workspace-level settings. HashiCorp recommends that Health assessments feature be **enabled for all workspaces**. This *global* setting is available under the main **Settings -> Health** page.

The Platform Team admin optionally can elect to enable Health Assessments on specific workspaces in the following circumstances:

- Rate limiting from the provider API is triggered - requiring reduction in API calls.
- Terraform Enterprise (Self-hosted) - If load in your Terraform Enterprise instance or its agents is too great and vertically scaling the compute layer is undesirable or impossible (TFE admins have the ability to adjust the frequency health assessments are performed)
- Remediating every drift can lead to an increase in operational tasks.

Terraform Enterprise administrators can modify their installation's assessment frequency and number of maximum concurrent assessments from the admin settings console.

We recommend that Workspace notifications is enabled and the Workspace Admin/Owners are notified (e.g via Slack, Email) when a workspace has a health assessment issue. This will help the relevant team take immediate action to resolve issues with the infrastructure.

9.1.1 Drift Resolution workflow

Once a drift is detected, the workspace can notify the application team and it is their responsibility to decide the best way to resole the drift.

The Platform team can also use HCP Terraform explorer to review which workspaces have drift and contact the application teams to resolve the drift.

On the organization's workspaces page, HCP Terraform displays a health warning status for workspaces with infrastructure drift or failed continuous validation checks.

On the right of a workspace's overview page, HCP Terraform displays a Health bar that summarizes the results of the last health assessment.

- The **Drift** summary shows the total number of resources in the configuration and the number of resources that have drifted.
- The **Checks** summary shows the number of passed, failed, and unknown statuses for objects with continuous validation checks.
- **Overwrite Drift**: For undesired changes, initiate a new Terraform plan and apply it to revert resources to their configuration-defined state.
- **Update Terraform Configuration**: If you want the drift's changes, modify your Terraform configuration to include the changes and push a new configuration version. This prevents Terraform from reverting the drift during the next apply.

In HCP Terraform, *refresh state* and *update Terraform configuration* are two distinct operations that play critical roles in infrastructure management, especially in the context of drift detection and remediation. Understanding the differences between these operations and knowing when to use each is vital for effective infrastructure as code practices.

1. **Refresh state**: Refreshing the state is the process of updating Terraform's internal state file to match the actual state of the infrastructure as it exists in the cloud or on-premises environments. This operation does not modify the infrastructure; instead, it updates Terraform's record (the state file) to reflect any changes that have occurred outside of Terraform's management. The primary purpose of refreshing the state is to ensure that Terraform's understanding

of the infrastructure is accurate. This accuracy is essential for identifying drift, the divergence between the desired state defined in Terraform configurations and the actual state of the infrastructure.

2. **Update Terraform configuration**: Updating a Terraform configuration version involves submitting updated Terraform configuration files to HCP Terraform and executing a plan and apply operation based on these updates. This process changes the infrastructure to match the desired state defined in the new configuration version.

    - **Update Terraform configurations**: Once drift is identified, update your Terraform configurations to either realign the infrastructure to the desired state or update the desired state to incorporate intentional changes made outside of Terraform.
    - **Review and apply changes**: Submit the updated configurations as a new configuration version in HCP Terraform. Take advantage of HCP Terraform's planning and review process to ensure all team members understand the changes and their impacts before applying them.
    - **Monitor and validate**: After applying the new configuration version, monitor the apply operation's outcome to ensure the intended changes were successfully implemented and validate that the infrastructure state aligns with the desired configuration.

Additional guidance

- **Continuous integration/continuous deployment (CI/CD)**: Integrate both the state refresh and apply operations into your CI/CD pipelines to facilitate continuous monitoring and timely remediation of drift.
- **Version control and code reviews**: Use version control systems for managing Terraform configurations. Implement a code review process for changes to Terraform files to ensure accuracy and intent of changes before they are applied.
- **Change management**: Incorporate these Terraform operations within your organization's change management framework to ensure changes are tracked, audited, and aligned with organizational policies and compliance requirements.
- **Documentation and training**: Document your workflow and operations, including state refresh and configuration updates, and provide training to your team. This ensures a consistent approach to managing and remediating drift across your organization.

For a detailed example, refer to the manage resource drift tutorial.

9.2 Continuous validation

We recommend that organizations use continuous validation for their workspaces especially to track the health of critical resources.

Continuous validation is the second component of the health check feature in HCP Terraform and TFE. The purpose of health checks is that by running regularly, they allow the infrastructure engineer to detect (and proactively fix) issues before they impair the next Terraform apply operation.

While drift detection flags out-of-band changes to the managed infrastructure that can affect a Terraform apply operation, continuous validation addresses use cases where more customizable detection rules are necessary. Some examples of those use cases are:

- To flag issues with infrastructure elements managed outside of the workspace, but that affect the health of this workspace's resources.
- To identify if cloud services or third-party tools have detected issues with the managed infrastructure resources.

Please review official documentation on continuous validation before implementing.

9.3 Why Continuous Validation?

Failed infrastructure changes can be costly to the organization as they may introduce project delays and could expose the organization to operational or security risks. Adopting continuous validation gives infrastructure teams advance notice of issues preventing successful changes in configuration. These issues can then be addressed and failed infrastructure changes avoided.

9.4 Best Practice Recommendation

When a new workspace is created to manage infrastructure, continuous validation should be enabled (either explicitly at the workspace level or implicitly at the organization level).

The infrastructure as code engineer should also include in the Terraform configuration code the necessary logic to validate important components of the infrastructure whose health may change over time and prevent a successful Terraform Apply run the next time an infrastructure configuration change is necessary.

In addition, if infrastructure changes do fail in the future because of a condition that was not checked, an engineer should update the Terraform configuration to incorporate this new validation. If applicable, this new pattern should be applied to existing infrastructure code and added to the checklist for future Terraform configurations.

Rule of thumb on which resources should have continuous validation applied:

1. Check the status of any critical resource that can fail (e.g VM)
    1. This is not necessary for certain resources such as S3 buckets which are native to the cloud provider.
2. Validity of resources such as certificates that have user defined time frames but whose failure can have an impact on the application stack.

9.5 Implementation guidance

In this section, we'll go over the steps and recommendations to implement continuous validation.

9.5.1 Requirements

Continuous validation requires the use of Terraform language features that aren't available in older versions of the runtime. The table below lists the language feature and the minimum version of the Terraform runtime that supports it.

Language feature	Version requirement	Useful links
Preconditions and postconditions	1.2 and later	Terraform 1.2 Improves Exception Handling and Updates to the CLI-driven Workflow Preconditions and postconditions
Check block	1.5 and later	Terraform 1.5 brings config-driven import and checks Checks with assertions

9.5.2 Permissions required

To configure continuous validation, you'll need the following permissions:

- To change organization health settings, you must be a member of the owners team.
- To change a workspace's health settings or trigger an on-demand health assessment, you must be an administrator for that workspace.

To view the continuous validation status:

- To view health status for a workspace, you need read access to that workspace.
- To view the status of all workspaces in the HCP Terraform Explorer, you need to be a member of the owners team or have the "View all workspaces" permission or better.

9.5.3 Enabling health assessments

Continuous validation being a component of the health assessment feature, to use continuous validation you must enable the feature.

You may enable health assessments at the organization level or at the workspace level, using the WebUI, the API or using Terraform code. Whenever possible we recommend using Terraform code to configure your HCP Terraform or Terraform Enterprise instance.

Enable health assessments at the ...	Using the WebUI	Using the API	Using Terraform
Organizati level	Managing Settings section	Organizations API	TFE provider: tfe_organization resource
Workspace level	Enable health assessments section	Workspaces API	TFE provider: tfe_workspace resource

Before adopting continuous validation, review your inventory of workspaces and specifically the Terraform version they are configured to use. Also review your code base and any Terraform required version constraint. If one or more workspaces do not meet the minimum requirements, they will not fully benefit from health assessments: Drift detection may be available (support was added in Terraform 0.15.4) but not the continuous validation health assessments.

If you decide to enable health assessments at the workspace level and you are using Terraform code to configure your HCP Terraform or Terraform Enterprise instance, you can use the flexibility of the Terraform language to selectively activate health assessments on workspaces that meet the requirements.

9.5.4 Defining custom assertions and checks

While continuous validation will evaluate preconditions, postconditions, and check blocks as part of an assessment, we recommend using check blocks for post-apply monitoring. Check blocks do not stop a Terraform run when they fail (contrary to preconditions and postconditions) which makes them ideal tools for monitoring.

Here is a list of useful resources to learn more about the check block:

- Checks (documentation)
- Use checks to validate infrastructure (tutorial)
- Health checks with Terraform Cloud continuous validation (demo)

Here is a list of useful resources showcasing examples using the check block:

- Ensure your AWS account is within budget (AWS)
- Check GuardDuty for Threats (AWS)
- Check for unused IAM roles (AWS)
- Check EKS Cluster Instance Health and Availability (AWS)
- Check for EC2 Stopped Instances (AWS)
- Check if a VM's is not running (Azure)
- Check if a Container App certificate will expire within a certain timeframe (Azure)
- Check if an App Service Function or Web App has exceeded its usage limit (Azure)
- Assert a VM is in a running state (GCP)
- Check if a certificate will expire within a certain timeframe (GCP)
- Validate the status of a Cloud Function (GCP)

9.5.5 Authentication considerations

Health checks require valid credentials in the workspace/run context - If you inject temporary credentials via external pipelines, which then are expired when health/drift runs happen, we rec-

ommend adopting "Dynamic provider credentials" which provide the same benefits of temporary credentials while being compatible with health checks

9.5.6 Viewing health status and getting notified

You can view the workspace(s) health status:

- From the workspace summary page (the **Health** panel on the right side of the interface).
- From the workspace continuous validation status page (from the workspace page, go to **Health** > **Continuous validation**).
- From the **Projects & Workspaces** view.
- From the HCP Terraform **Explorer**.

You can also configure notifications to get alerted on a number of events, including:

- When a continuous validation check returns unknown or failed.
- When a health assessment cannot be completed successfully.

Notifications can be sent to a number of different supported destinations:

- Email
- Slack
- Microsoft Teams
- Webhooks (most flexible)

This feature is a workspace-level configuration, which means that you'll need to configure it for every workspace that you need to monitor and get a notification. If you are using Terraform code to configure your HCP Terraform or Terraform Enterprise instance, you can use the flexibility of the Terraform language to easily configure notifications on workspaces that need it.

10 Terraform Workspace Notifications

Workspace notifications are a mechanism to notify external systems of significant events happening with an HCP Terraform workspace. We recommend that at the minimum a team specific channel (e.g Microsoft Teams/Slack) or email group is created for each admin team and critical notifications are sent to those channels.

Notification events are broken down into two categories: workspace events and run events.

Workspace events that can trigger a notification are:

- Check failed: When a continuous validation check returns unknown or failed.
- Drift detected: Every time drift is detected on this workspace.
- Health assessment errored: When a health assessment cannot complete successfully.
- Auto-destroy reminder: Sends a reminder 12 and 24 hours before an auto-destroy run.
- Auto-destroy results: The results of an auto-destroy run.

All of the above events are considered critical. If you consider the amount of notifications generated is too many, consider "Check failed", "Health assessment errored" and "Auto-destroy reminder" at the minimum to be important where admins are notified.

Run events that can trigger a notification are:

- Created: Every time a run is created and enters the "Pending" state.
- Planning: When a run acquires the lock and starts to execute.
- Needs attention: Human decision required. When a plan has changed and is not auto-applied, or requires a policy override.
- Applying: After a plan is confirmed or auto-applied.
- Completed: When the run has been completed successfully.
- Errored: If the run has terminated early due to error or cancelation.

Needs attention and Errored are considered critical run events and notifications should be enabled for those at the minimum.

The notification includes information about the event that provides more context to the external system.

10.1 Implementation guidance

Notifications can be configured using the WebUI, the API, or using Terraform configuration (tfe_notification_configuration). HashiCorp recommends using the Terraform TFE provider to configure notifications on your HCP Terraform or Terraform Enterprise instance as part of the project/workspace creation process.

10.1.1 Permissions required

To configure notifications on a workspace a user needs Admin access to that workspace.

10.1.2 Notification strategy

When designing your notification strategy, consider the following factors:

- Choose the Appropriate Destination: Pick communication channels that fit your team's workflow. Slack is popular, but you can also use email or other platforms based on your needs.
- Granular Notifications: Avoid broad notifications that can cause alert fatigue. Focus on critical events and changes that need immediate attention. For example, you might want alerts only for failed runs or significant changes to critical resources.
- Integration with Incident Management: Integrate with incident management tools to streamline responses, ensuring alerts lead to actionable items and quick incident resolution.

10.1.3 Maintenance

Once notifications are in use, they do not require a lot of work, but you should still consider the practices below:

- Regular Reviews: Periodically review your notification settings and adjust them based on changing infrastructure needs and team feedback.
- Test and Validate: When making changes or adding new notifications, test your notification setup to make sure it works. Trigger events manually and check if notifications are received in the chosen destination.
- Monitor and Adjust: Continuously monitor and solicit feedback on the effectiveness of your notifications. Use this information to tweak the configuration to reduce noise and improve relevance.

11 No-code provisioning

Note

No-code modules are published, allow-listed, and governed by platform operators — see the Administration Guide: No-code provisioning management.

12 Ephemeral workspaces

The ephemeral workspace feature allows teams to set a timeout for automatically destroying resources managed by a workspace. The delay is either fixed (a specific day and time) or relative (after a period of inactivity). We recommend that you consider this feature for temporary and development workspaces as a way to decrease cloud wastel

With this feature, once the configured timeout (or inactivity period) is reached, all the resources managed by the workspace are destroyed, but the workspace and its configuration are still available. If the infrastructure is needed again at a later time, the user can simply initial a Terraform apply operation to recreate the environment.

Please review theofficial documentation for more details on this capability.

12.1 Implementation guidance

Tip

HashiCorp recommends using ephemeral workspaces in lower infrastructure environments, such as development environments or sandboxes. We also recommend that this feature is enabled as part of the workspace provisioning workflow.

12.1.1 Roles and responsibilities

Implementing ephemeral workspaces requires collaboration between several key roles:

- Platform Engineers are responsible for configuring the TTL policies, destroy schedules, and notifications in HCP Terraform. They ensure that environments are efficiently managed and

align with best practices for performance, cost, and security. Platform engineers should work closely with other teams to define standards for non-production environments.
- Development and QA Teams consume ephemeral workspaces for tasks such as feature development and testing. Although they don't configure the environments, they need to coordinate with platform engineers to ensure that TTL settings align with their workflows, avoiding destruction during critical work periods.
- Security Teams oversee compliance with security standards, ensuring that workspace destruction processes adhere to policies and that no sensitive data remains after resources are destroyed.

12.1.2 Permissions requirements

The following permissions are involved in setting up ephemeral workspaces:

- Administration permissions on the workspace (workspace-level).
- Project Admin on the project (project-level).
- Manage all projects organization-level permission.

12.1.3 Configuring at scale

While it is possible to set up ephemeral workspaces on an individual workspace level, this is not the recommended approach. Instead you should consider one of the following approaches:

- Provisioning of ephemeral workspaces as code.
- Self-service provisioning of ephemeral workspaces.

Provisioning ephemeral workspaces as code

In this approach, the platform team leverages the Terraform configuration used to configure HCP Terraform or Terraform Enterprise (see the Landing zones section applied to HCP Terraform configuration) to provision workspaces with ephemeral workspace attributes set (see tfe_workspace). The standard pattern for such a workspace can be codified in a Terraform module.

This approach works if the platform team is in charge of the deployment of such workspaces, or if developers are sufficiently knowledgeable about HCP Terraform or Terraform Enterprise to be delegated the task of provisioning ephemeral workspaces in projects they have been granted administrative access to.

Self-service provisioning of ephemeral workspaces

In this approach, the platform team sets up one (or more projects) dedicated to ephemeral workspaces, and a Terraform no-code module that provisions the ephemeral workspace in the dedicated project.

The project's configuration must set a default timeframe workspaces to automatically destroy their infrastructure. This can be done using the WebUI or automated using the HCP Terraform API. You should also ensure that the workspaces have the necessary permissions to provision infrastructure, either by using dynamic provider credentials (preferred), IAM roles, or static credentials.

> Note
>
> Self-service provisioning can also be implemented using HCP Terraform integration with ServiceNow using a no-code flow (see Terraform ServiceNow Service Catalog Integration Setup Instructions).

12.1.4 Setting up notifications

We recommend that notifications at the workspace level are set up so that development teams are informed of the following events related to ephemeral workspaces:

- Auto-destroy reminder (send a reminder 12 and 24 hours before an auto-destroy run)
- Auto-destroy results (the results of an auto-destroy run)

Please refer to the notification section for more details.

13 Changelog

13.1 2026-08

- Restructured legacy Terraform HVD content into the HVD 2.0 Terraform User Guide as part of the Installation / Administration / User guide migration.

13.2 2026-01

- Update meta descriptions

- Updated credits and contributors

13.3 2025-07

- Fix broken image references

13.4 2025-06

- Use consistent guide names and descriptions
- Standardized operating guide naming conventions

13.5 2024-09

- GA Release (Scaling Operating Guide)

13.6 2024-04

- GA Release (Standardization Operating Guide)

14 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adam Cavaliere
- Adeel Ahmad
- Alex Basista
- Amy Brown
- Anna Chen
- Brandon Ferguson
- Chris Steinmeyer

- Chris Stella
- CJ Obermaier
- Emmanuel Rousselle
- Fraser Pollock
- James Purcell
- Jenna Degaust
- Judith Malnick
- Kalen Arndt
- Kranthi Katepalli
- Kyle Seneker
- Mark Lewis
- Michelle Roberts
- Mike Augustine
- Monica Miller
- Nada Elkelani
- Prakash Manglanathan
- Purna Mehta
- Randy Keener
- Richard Russell
- Salil Subbakrishna
- Sean Doyle
- Sean Walker
- Shriram Rajaraman
- Shyam Chaganti
- Simon Lynch
- Yogeshprabhu Jagadeesan
- Zachary Isom
- Zeeshan Ratani
- Zeshan Ratani